

Video Encoding with PyAV

<https://drive.google.com/drive/folders/1ulW5L-blsEDjy-0fee9NfzSRNm6NTuTI?usp=sharing>

Input video link:

<https://drive.google.com/file/d/1yai-ZvczS1x4CsVDeaAMCQ4B5w53lqbN/view?usp=sharing>

```
Frames per second (FPS): 29.97002997002997
Total frames: 640
Analyzing frames and categorizing them...
Frame types count:
I-frame: 43
P-frame: 299
B-frame: 298
Other: 0
```

From frames_output folder:

-> **I-frame numbers:** 0, 15, 30, 45, ..., 630 (every 15th frame is an I-frame).

-> The **GOP (Group of pictures) size** is the difference between consecutive I-frames = 15 frames.

We confirmed it by writing python code to check how many frames were present in each GOP.

Below is the output:

[illegible]

Encode the videos using pyAV in three different configurations -- (i) only with I-frames, (ii) with a single I and rest P-frames, and (iii) single I and rest interleaved P and B frames. Do it for H.264 and H.265. Then compare the sizes of the videos created

H.264 codec

(i) Encoding the video using only I-frames.

Video Input: A sample video was chosen for encoding, and we used PyAV to read the video file.

Set the keyframe interval to 1, ensuring all frames would be encoded as I-frames.

```
Video Stream Attributes:
Width: 1920
Height: 1080
Sample Aspect Ratio: 1
Time Base: 1/30000
Codec: h264
Video with only I-frames has been saved to /kaggle/working/video_only_i_frames.mp4
```

```
PS D:\Arani Sir IP> ffprobe -v error -select_streams v:0 -show_entries frame=pict_type -of
csv=print_section=0 video_only i_frames.mp4
```

```
PS D:\Arani Sir IP> ffprobe -v error -select_streams v:0 -show_entries frame
y_i_frames.mp4
I,
I,
I
I
I
I
I
I
I
I
I
I
I
I
```

Output: The resulting encoded video is saved as `output_video_single_i_and_p_frames.mp4`.

```
Video with a single I-frame followed by P-frames has been saved to /kaggle/working/output_video_single_i_and_p_frames.mp4
Number of I-frames: 1
Number of P-frames: 639
```

```
PS D:\Arani Sir IP> ffprobe -v error -select_streams v:0 -show_entries frame=pict_type -of
csv=print_section=0 output_video_single_i_and_p_frames.mp4
```

[illegible]

(iii) Encoding the video using single I and rest interleaved P and B frames.

Output: The resulting encoded video is saved as `single_i_interleaved_p_b_frames`.

```
Video with a single I-frame followed by interleaved P and B frames has been saved to /kaggle/working/single_i_interleaved_p_b_frames.mp4
Number of I-frames: 1
Number of P-frames: 320
Number of B-frames: 319
```

```
ffprobe -v error -select_streams v:0 -show_entries frame=pict_type -of csv=print_section=0
single i interleaved p b frames.mp4
```

```
PS D:\Arani Sir IP> ffprobe -v error -select_streams v:0 -show_entries frame=pict_type -of csv=pri
interleaved_p_b_frames.mp4
I,
B
P
B
P
B
P
B
P
B
P
P
```

Task: Try with multiple crf value for each encoding techniques and compare the sizes.

Below are the crf values that we considered- [0, 10, 18, 23, 28, 35, 40]

Below is the table comparing different sizes (in MB) for different crf values for each of the encoding techniques:

Encoding techniques (H.264)/ crf values	0 (in MB)	10 (in MB)	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	592.80	189.57	91.18	55.11	30.10	12.19	6.78
Single I and rest P-frames	541.19	157.85	52.53	25.26	12.00	4.96	3.19
Single I and interleaved B and P-frames	361.66 (no B frames)	64.10	23.64	13.29	7.32	3.55	2.28

The CRF 0 setting is typically avoided if you want B-frames because B-frames are avoided in lossless encoding. A CRF of 18-23 provides a good balance between quality and compression with interleaved frames.

Task: Encode the videos using pyAV in three different configurations for H.265

Encoding techniques (H.265)/ crf values	0 (in MB)	10 (in MB)	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	327.42	98.09	43.40	27.61	17.58	8.84	5.15
Single I and rest P-frames (forced gop = 15)	226.06	45.95	12.44	6.05	3.31	1.52	0.84
Single I and interleaved B and P-frames (gop = 15)	172.60	34.94	10.28	5.34	3.01	1.39	0.78

Note: For single I and rest P-frames, I got gop size=30. For "**Single I + rest P-frames**", H.265 **naturally extends the GOP size to 30** because it benefits more from longer GOPs (better compression, same quality).

Task Decode the input video and re-encode it with specific configurations.

Decode Part:

1. The **decode** step reads frames from the input video, decompressing them into raw image data.
2. This is done using `av.open(input_file)`, which loads the video file.
3. We extract the **video stream** (`input_stream`) and iterate over each frame using:
`for i, frame in enumerate(input_container.decode(video=0)):`
4. The frames remain in **uncompressed format** in memory temporarily.

Encode Part:

1. The **encoding** step takes the raw frames, compresses them into a new format, and writes them to the output file.
2. Encoding is done using `output_container.add_stream(codec, rate=frame_rate)`, where `codec = 'h264'` (for H.264) and `codec = 'hevc'` (for H.265)
3. The **GOP (Group of Pictures)** structure is applied, determining I-, P-, and B-frames:
`if i % gop_size == 0:`
`frame.pict_type = av.video.frame.PictureType.I # I-frame`
`elif b_frames > 0 and i % 2 == 0:`
`frame.pict_type = av.video.frame.PictureType.B # B-frame`
`else:`
`frame.pict_type = av.video.frame.PictureType.P # P-frame`
4. This ensures **one I-frame per GOP**.
5. Finally, frames are **encoded and written** to the output file using:
`for packet in output_stream.encode(frame):`
`output_container.mux(packet)`
6. This process happens on-the-fly to avoid excessive memory usage.

Output for H.264:

```
Encoding output_i_frames_crf_18.mp4...
Encoding output_i_frames_crf_23.mp4...
Encoding output_i_frames_crf_28.mp4...
Encoding output_i_frames_crf_35.mp4...
Encoding output_i_frames_crf_40.mp4...
Encoding output_i_p_frames_crf_18.mp4...
Encoding output_i_p_frames_crf_23.mp4...
Encoding output_i_p_frames_crf_28.mp4...
Encoding output_i_p_frames_crf_35.mp4...
Encoding output_i_p_frames_crf_40.mp4...
Encoding output_i_p_b_frames_crf_18.mp4...
Encoding output_i_p_b_frames_crf_23.mp4...
Encoding output_i_p_b_frames_crf_28.mp4...
Encoding output_i_p_b_frames_crf_35.mp4...
Encoding output_i_p_b_frames_crf_40.mp4...
```

Output for H.265

```
encoded 2245 frames in 437.77s (5.13 fps), 2996.10 kb/s, Avg QP:36.45
Encoding D:\Arani Sir IP\town_5\H.265\only I-frames\output_I_frames_crf_40.mp4...
x265 [info]: HEVC encoder version 3.5+1-f0c1022b6
x265 [info]: build info [Windows][GCC 14.2.0][64 bit] 8bit+10bit+12bit
x265 [info]: using cpu capabilities: MMX2 SSE2Fast LZCNT SSSE3 SSE4.2 AVX FMA3 BMI2 AVX2
x265 [info]: Main Intra profile, Level-4 (Main tier)
x265 [info]: Thread pool created using 8 threads
x265 [info]: Slices : 1
x265 [info]: frame threads / pool features : 3 / wpp(17 rows)
x265 [info]: Coding QT: max CU size, min CU size : 64 / 8
x265 [info]: Residual QT: max TU size, max depth : 32 / 1 inter / 1 intra
x265 [info]: ME / range / subpel / merge : hex / 57 / 2 / 3
x265 [info]: Lookahead / bframes / badapt : 0 / 0 / 0
x265 [info]: b-pyramid / weightp / weightb : 0 / 0 / 0
x265 [info]: References / ref-limit cu / depth : 1 / off / on
x265 [info]: AQ: mode / str / qg-size / cu-tree : 2 / 1.0 / 32 / 0
x265 [info]: Rate Control / qCompress : CRF-40.0 / 0.60
x265 [info]: tools: rd=3 psy-rd=2.00 early-skip rskip mode=1 signhide tmvp
x265 [info]: tools: b-intra strong-intra-smoothing lslices=6 deblock sao
x265 [info]: frame I: 2245, Avg QP:41.56 kb/s: 1692.51
x265 [info]: consecutive B-frames: 100.0%
```

Video link: [dataset_feb3](#)

H.264

Encoding techniques/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	522.10	301.22	164.96	68.20	35.22
Single I and rest P-frames	202.51	104.81	54.09	23.30	12.68
Single I and interleaved B and P-frames	190.61	100.53	53.32	23.29	12.76
Single I and interleaved 3 B's and single P-frames	180.28	96.95	52.73	23.30	12.78

H.265

Encoding techniques/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	533.62	317.83	182.71	81.83	44.21
Single I and rest P-frames	132.11	65.30	32.82	13.25	6.84
Single I and interleaved B and P-frames	116.52	58.37	29.62	11.93	6.12
Single I and interleaved 3 B's and single P-frames	109.48	55.26	28.41	11.50	5.90

Video link: [dataset_town_2](#)

H.264

Encoding techniques (H.264)/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	256.77	155.18	93.31	44.13	24.12
Single I and rest P-frames	119.32	64.02	34.90	15.90	9.23
Single I and interleaved B and P-frames	112.86	60.90	33.73	15.66	9.17
Single I and interleaved 3 B's and single P-frames	109.16	59.36	33.17	15.52	9.12

Instead of crf, change qp.

H.265

Encoding techniques/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	256.26	160.62	100.76	51.45	29.33
Single I and rest P-frames	84.21	44.31	23.92	10.37	5.55
Single I and interleaved B and P-frames	75.59	40.28	22.01	9.60	5.12
Single I and interleaved 3 B's and single P-frames	70.96	38.08	20.96	9.20	4.90

Video link: [dataset_town3](#)

H.264

Encoding techniques (H.264)/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	200.94	116.43	68.06	32.47	18.84
Single I and rest P-frames	112.17	55.89	28.20	12.22	7.45
Single I and interleaved B and P-frames	106.57	52.96	27.31	12.11	7.37

H.265

Encoding techniques/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	192.23	115.46	71.05	37.13	22.65
Single I and rest P-frames	80.43	39.19	19.56	8.01	4.31
Single I and interleaved B and P-frames	69.59	34.26	17.49	7.27	3.93

Video link: [dataset_town4](#)

H.264

Encoding techniques (H.264)/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	158.22	90.88	51.87	24.05	13.66
Single I and rest P-frames	62.24	33.25	17.87	7.74	4.46
Single I and interleaved B and P-frames	60.80	32.26	17.47	7.67	4.43

H.265

Encoding techniques/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	146.78	88.35	54.16	27.65	17.10
Single I and rest P-frames	38.54	19.70	10.50	4.45	2.41
Single I and interleaved B and P-frames	34.02	17.64	9.51	4.04	2.20

Video link: [dataset_town5](#)

H.264

Encoding techniques (H.264)/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	325.18	190.59	109.37	50.81	28.48
Single I and rest P-frames	128.86	72.63	39.25	16.55	9.53
Single I and interleaved B and P-frames	125.27	70.07	38.11	16.33	9.38

H.265

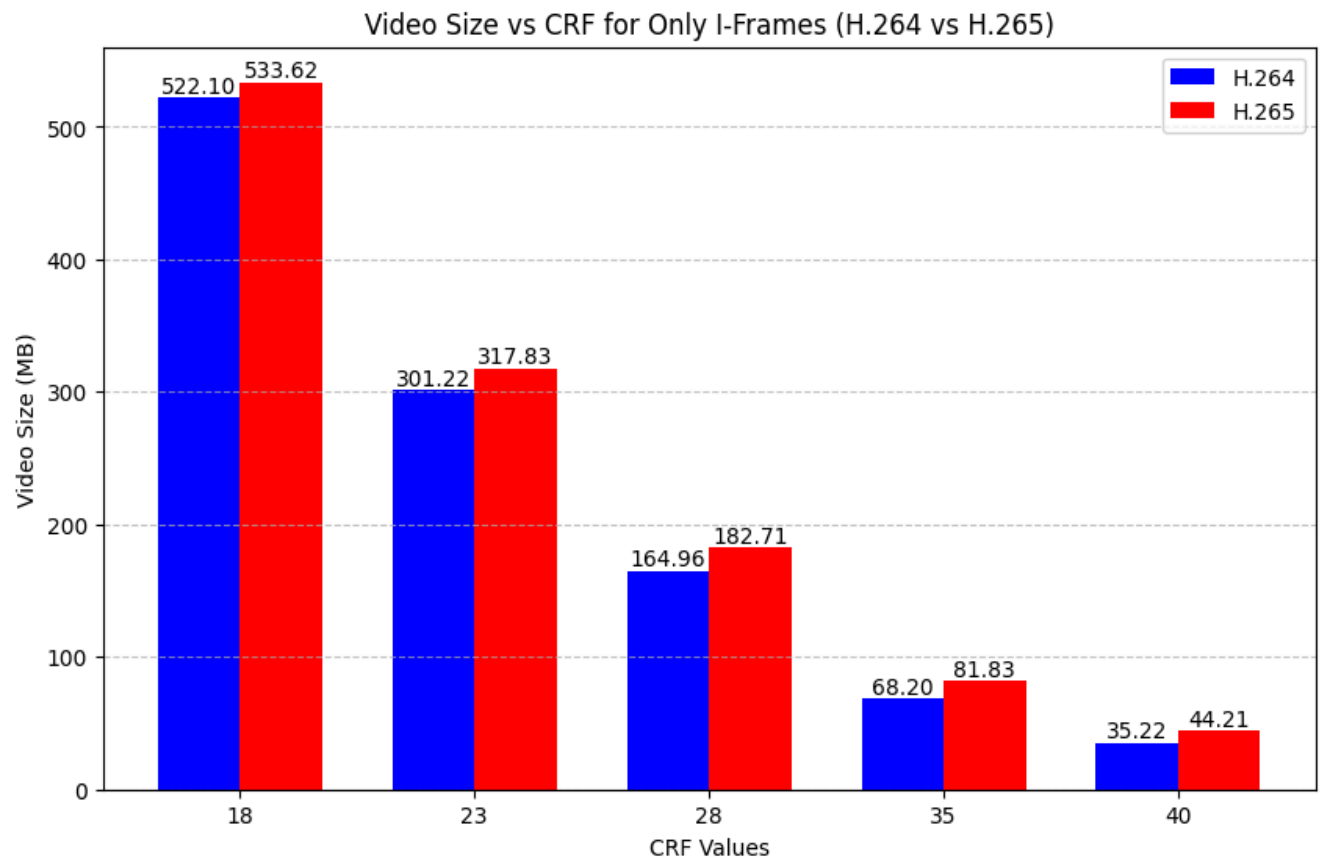
Encoding techniques/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Only I-frames	307.51	189.84	117.58	59.84	36.02
Single I and rest P-frames	89.25	48.07	26.00	10.87	5.85
Single I and interleaved B and P-frames	78.87	42.65	23.20	9.76	5.25

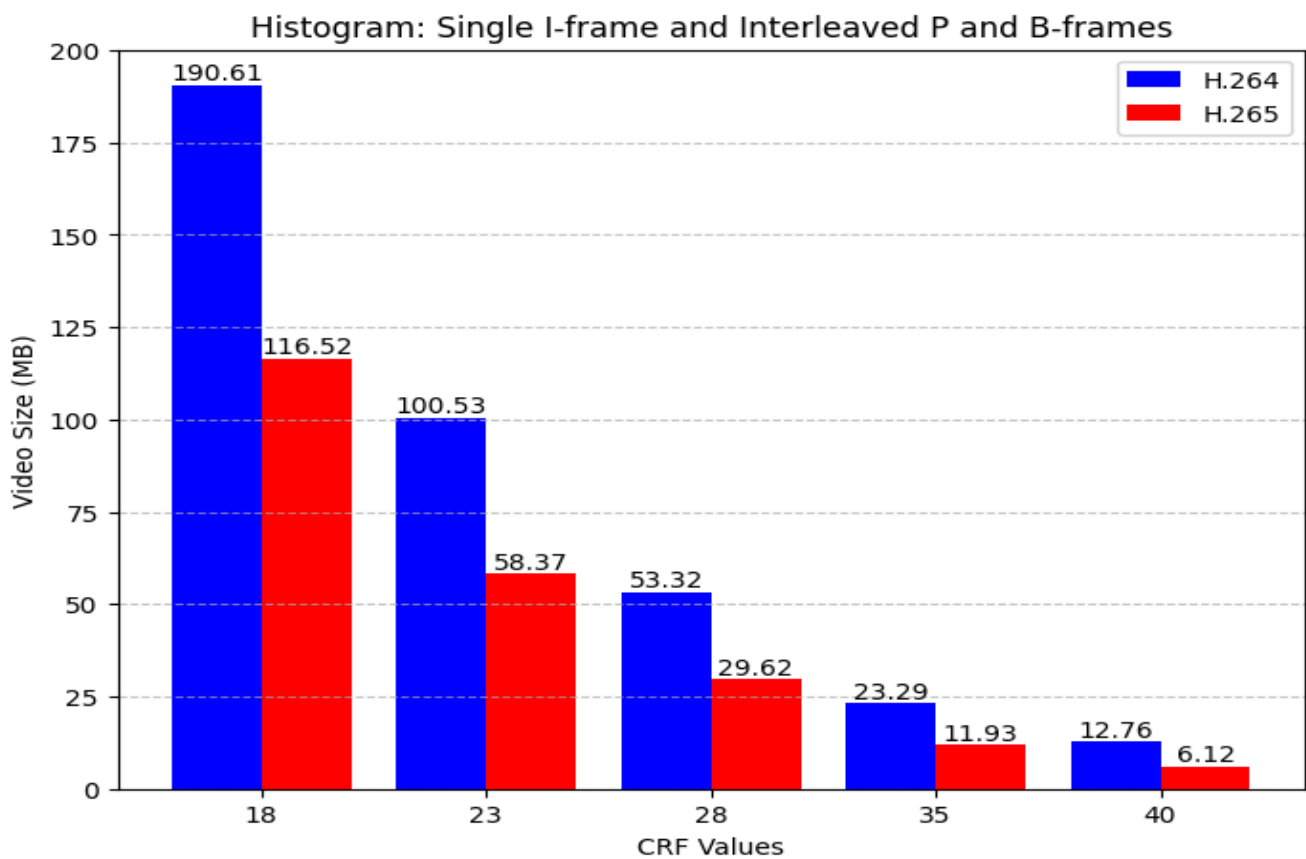
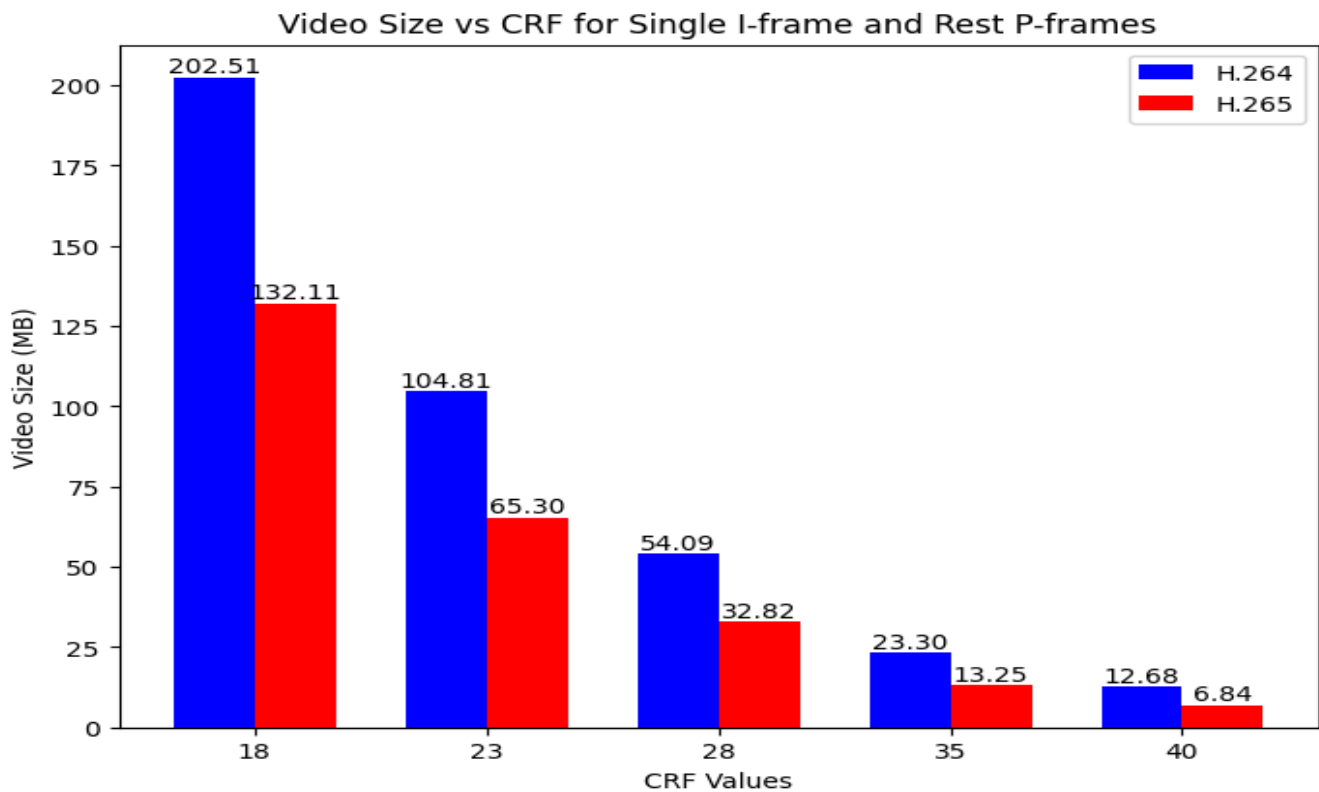
Task Create a descriptive histogram for different frame types:

- Only I-frame
- Single I-frame rest P-frames
- Single I-frame with P and B frames

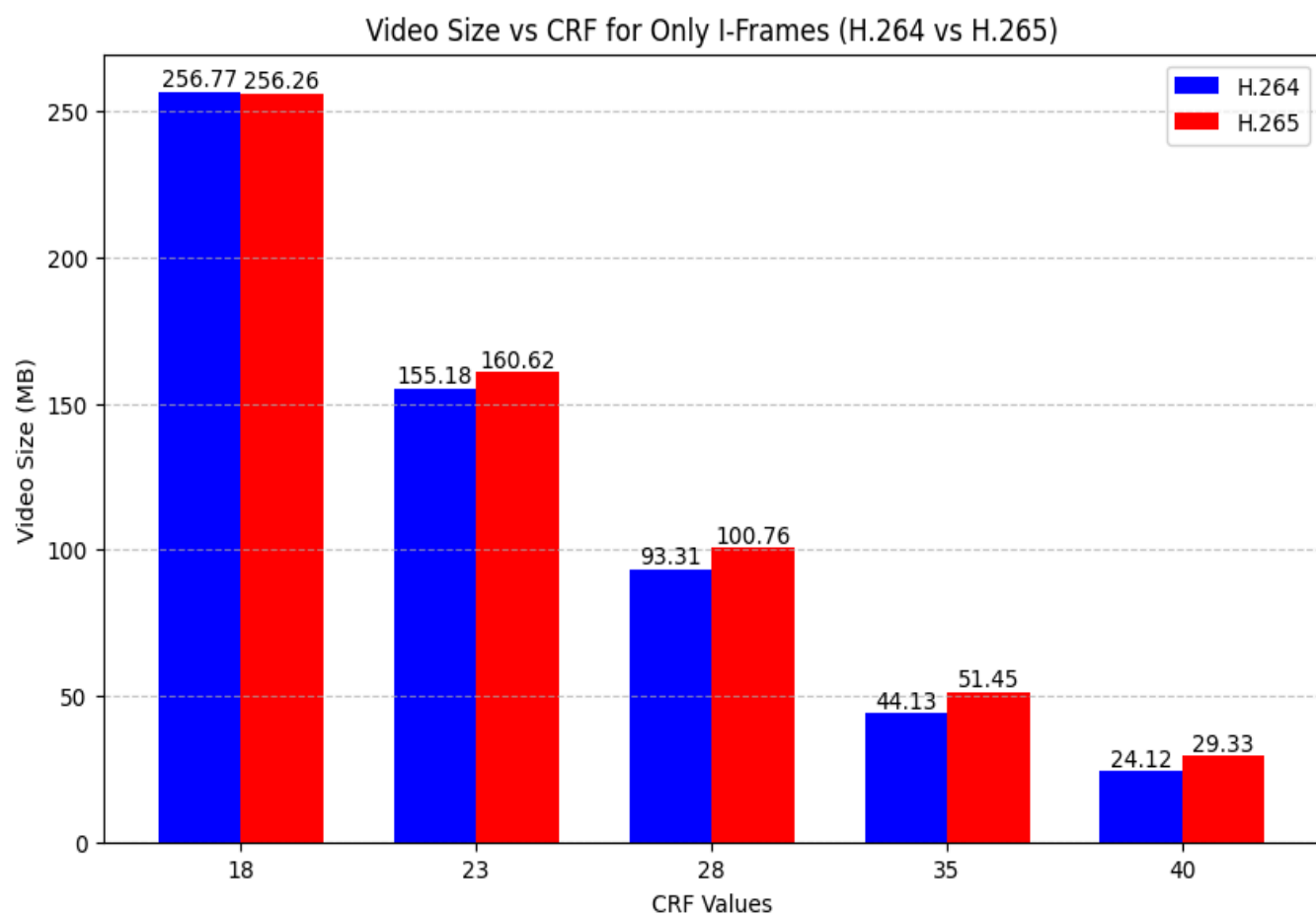
- X-axis: CRF values and Y-axis: Video size

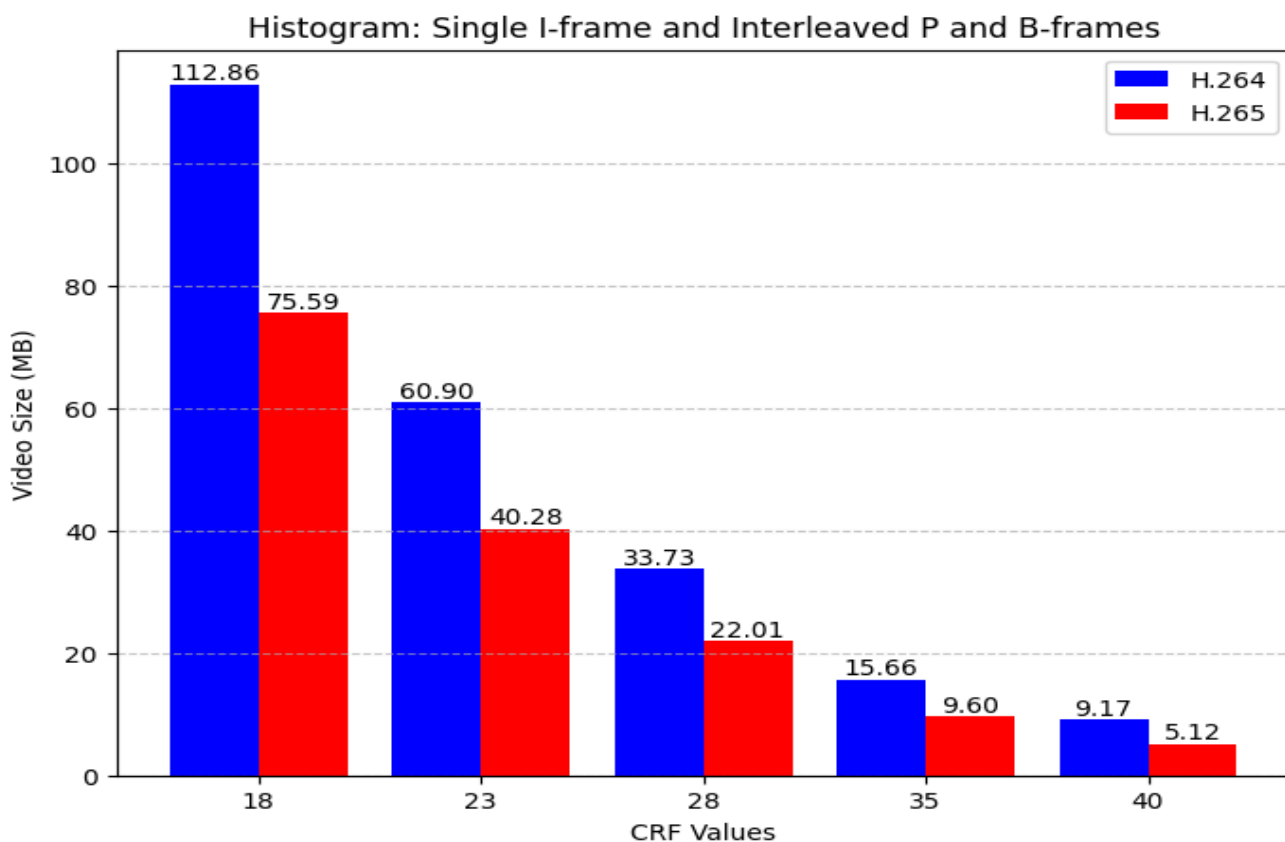
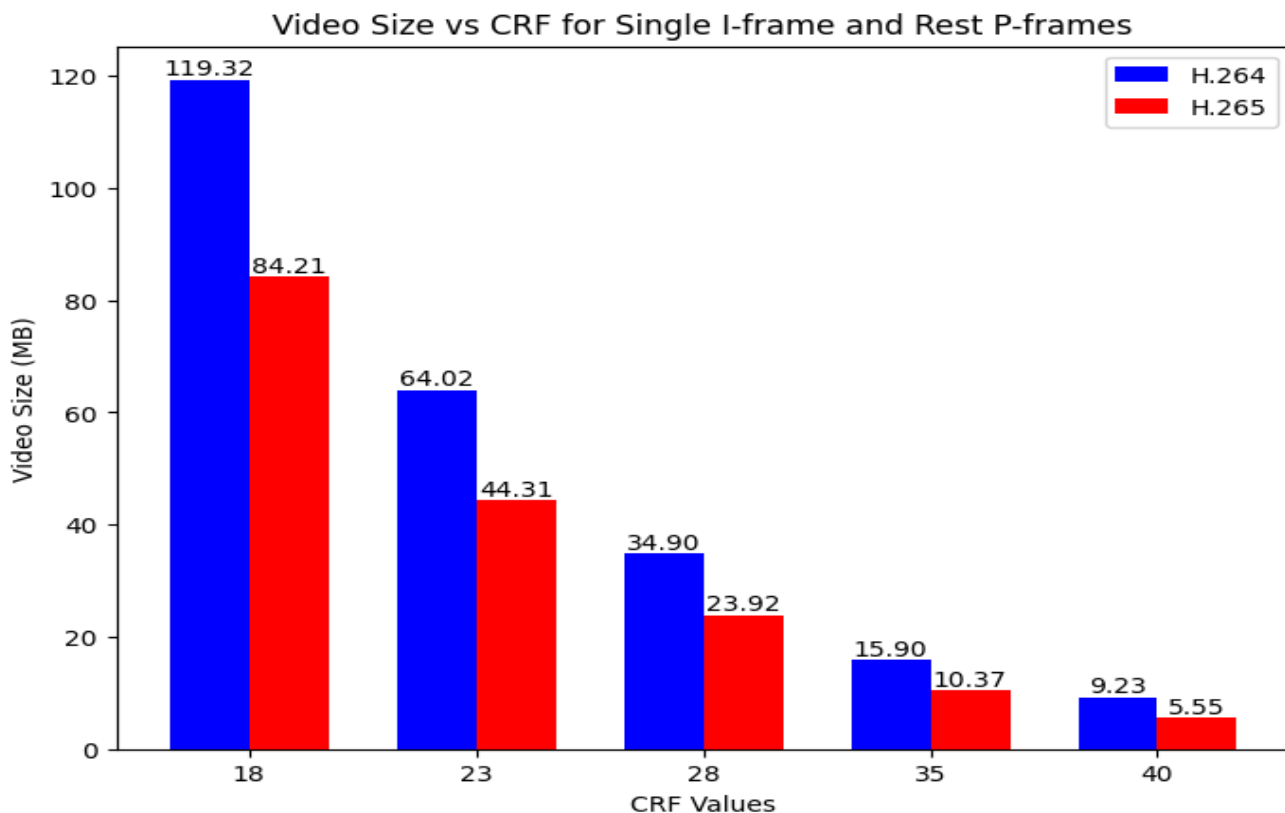
Video link: [dataset_feb3](#)



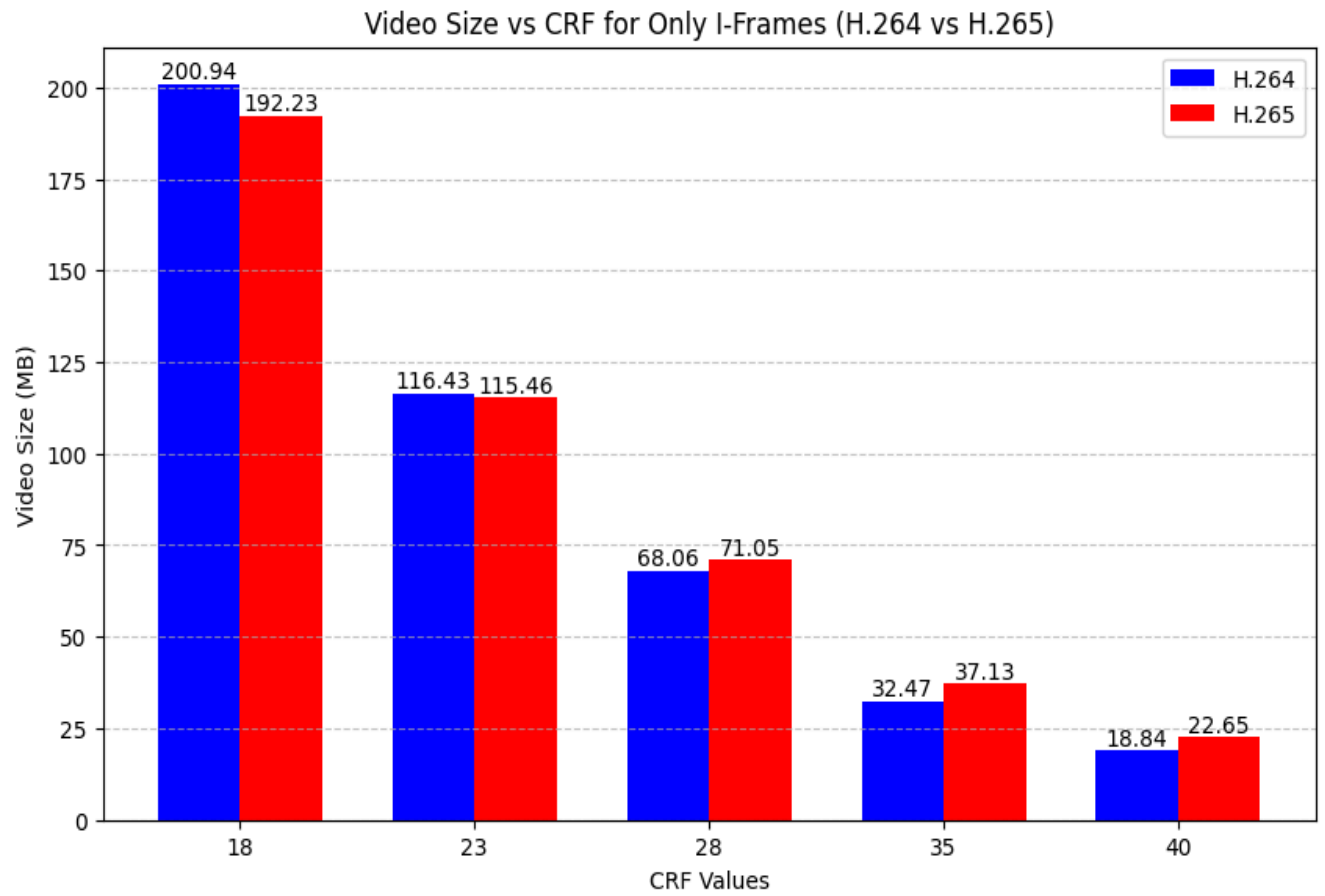


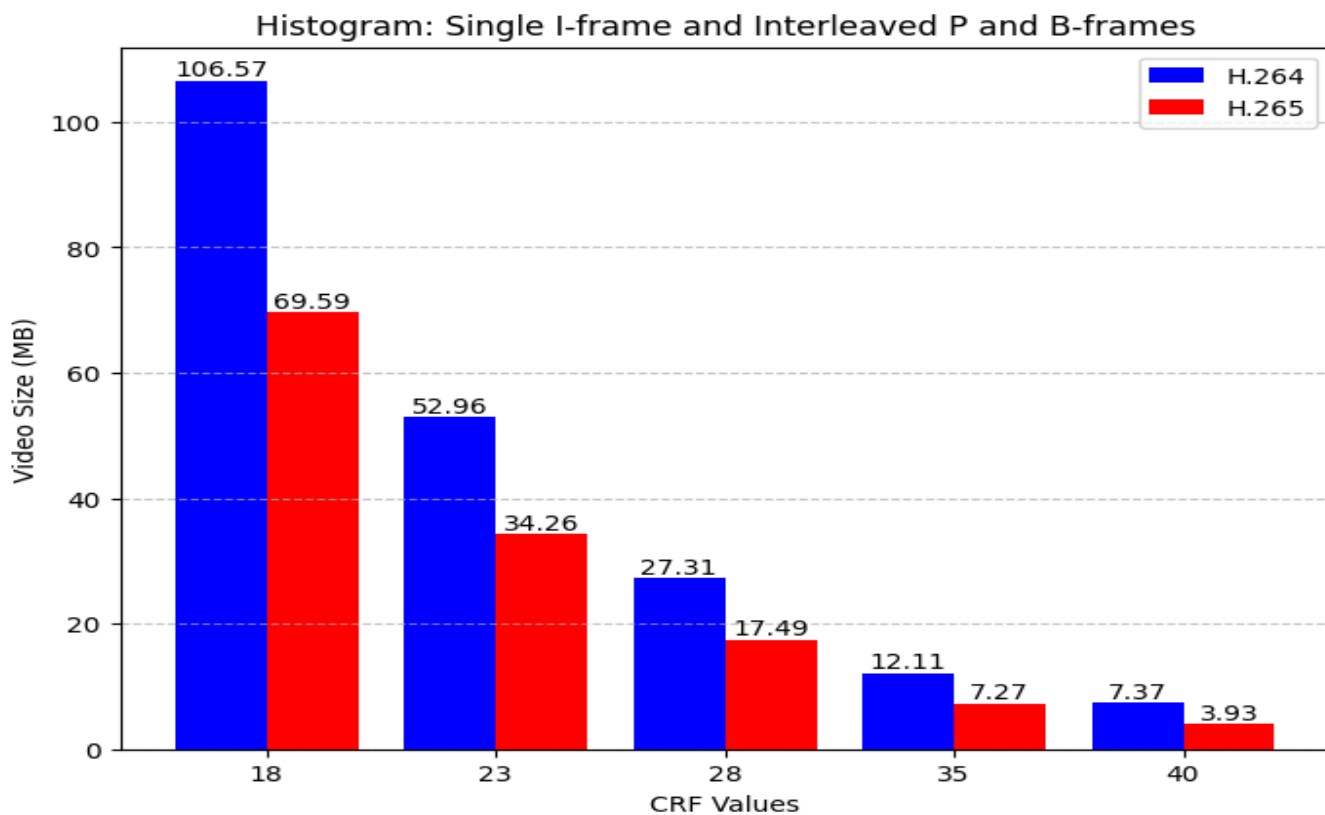
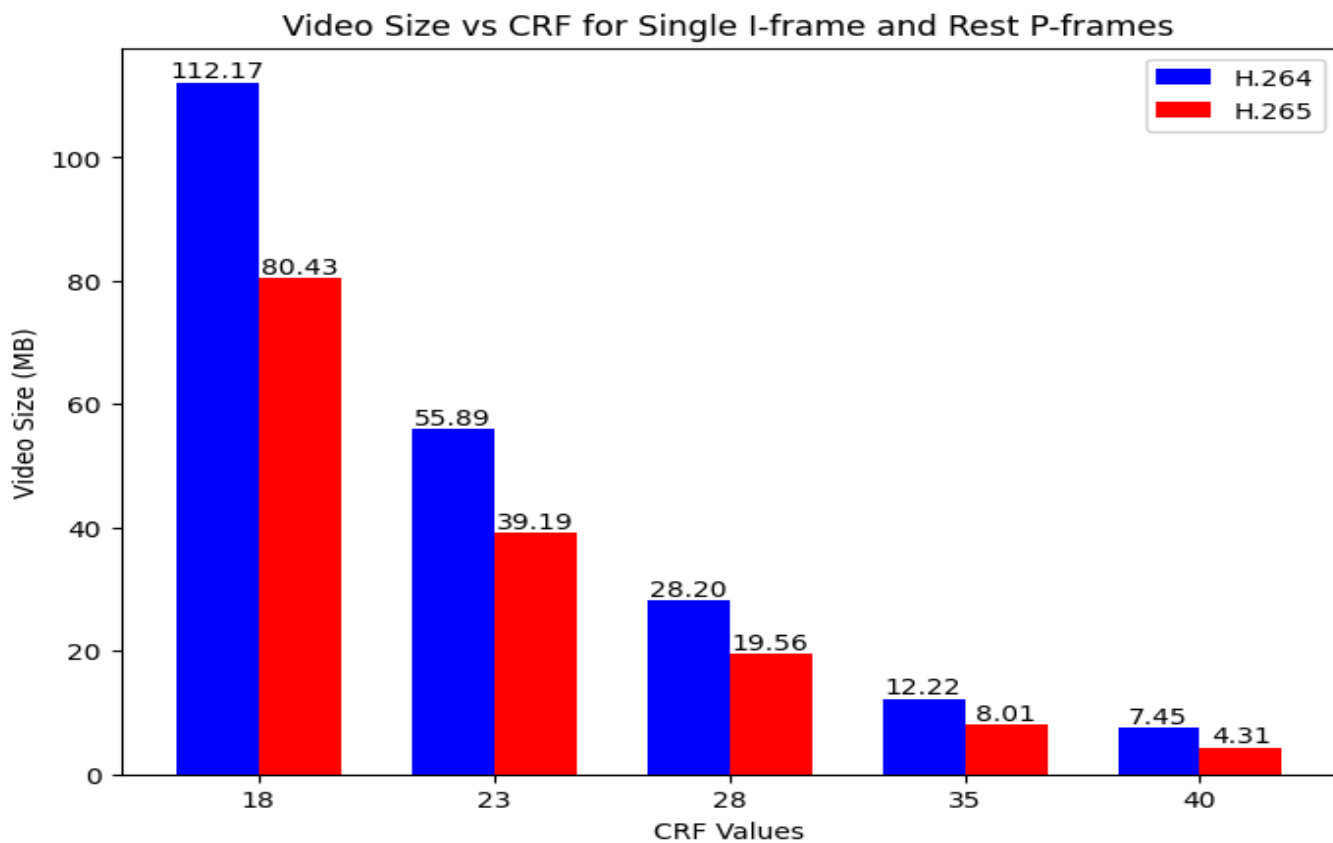
Video link: [dataset town 2](#)



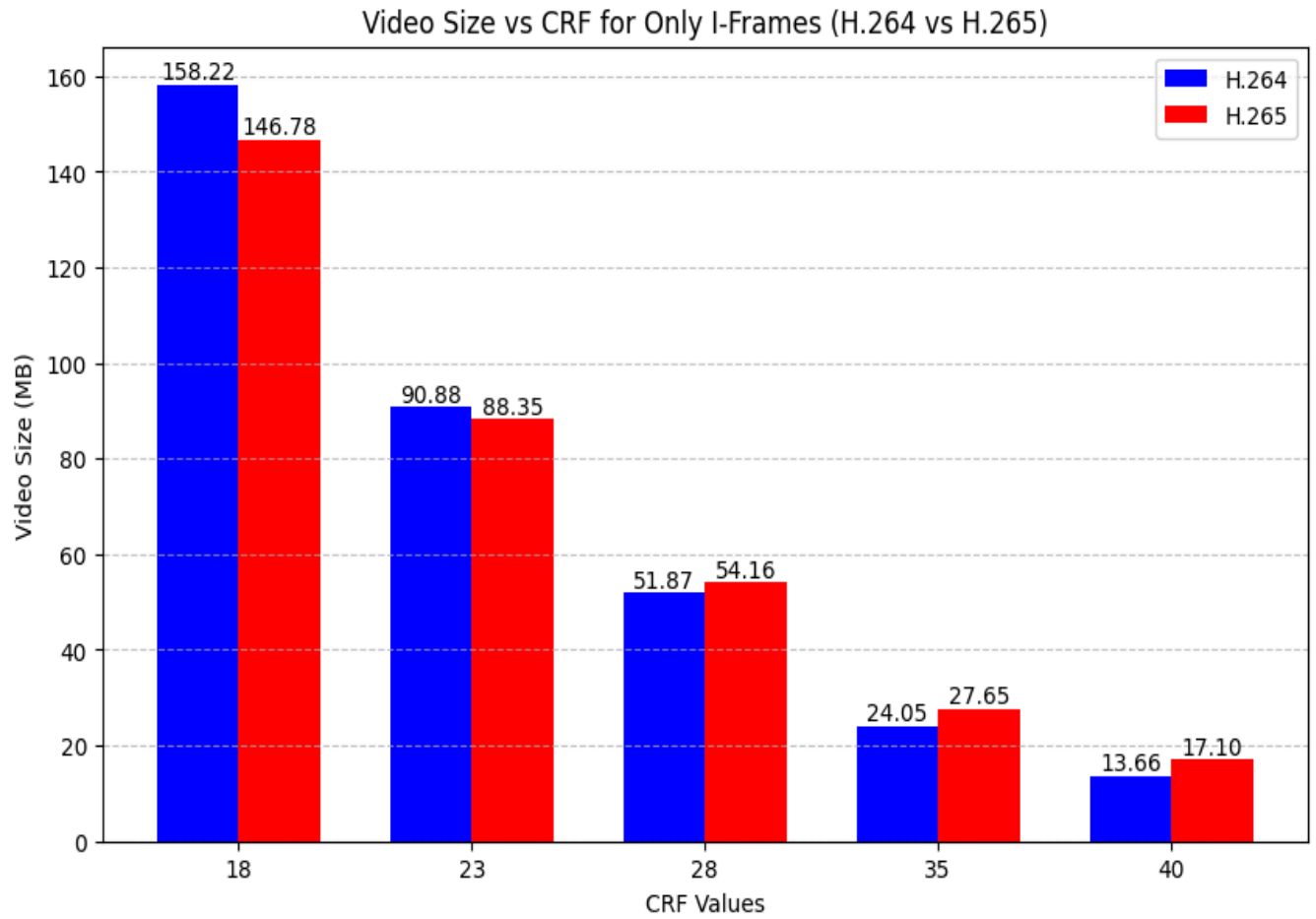


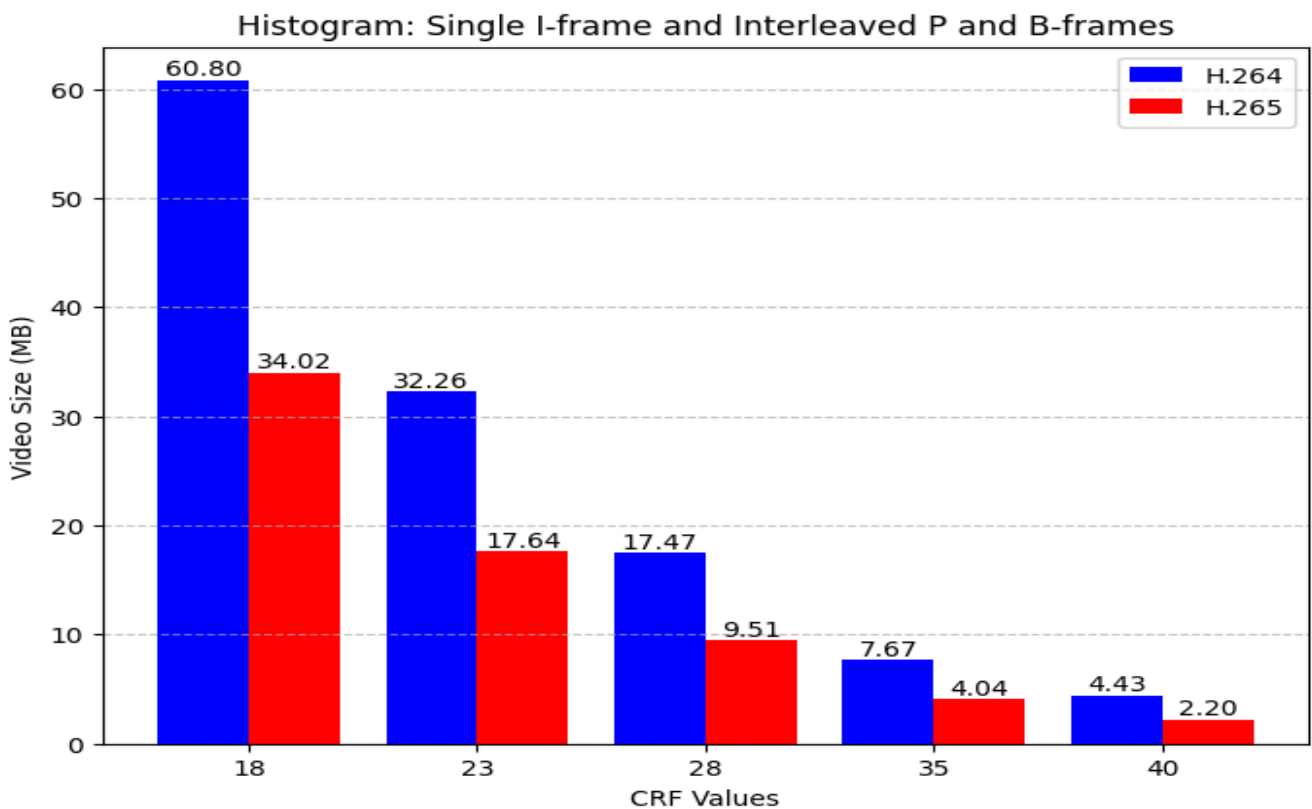
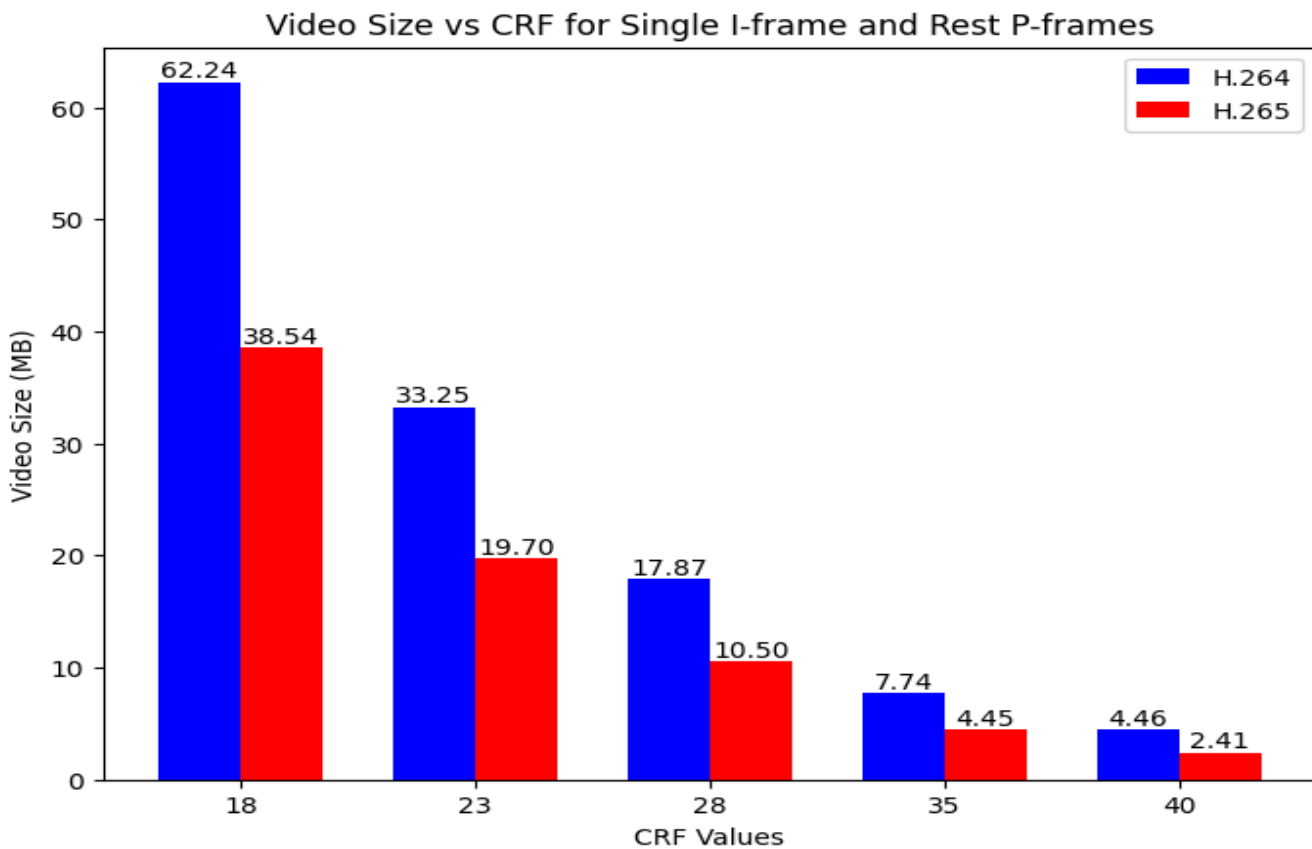
Video link: [dataset_town3](#)



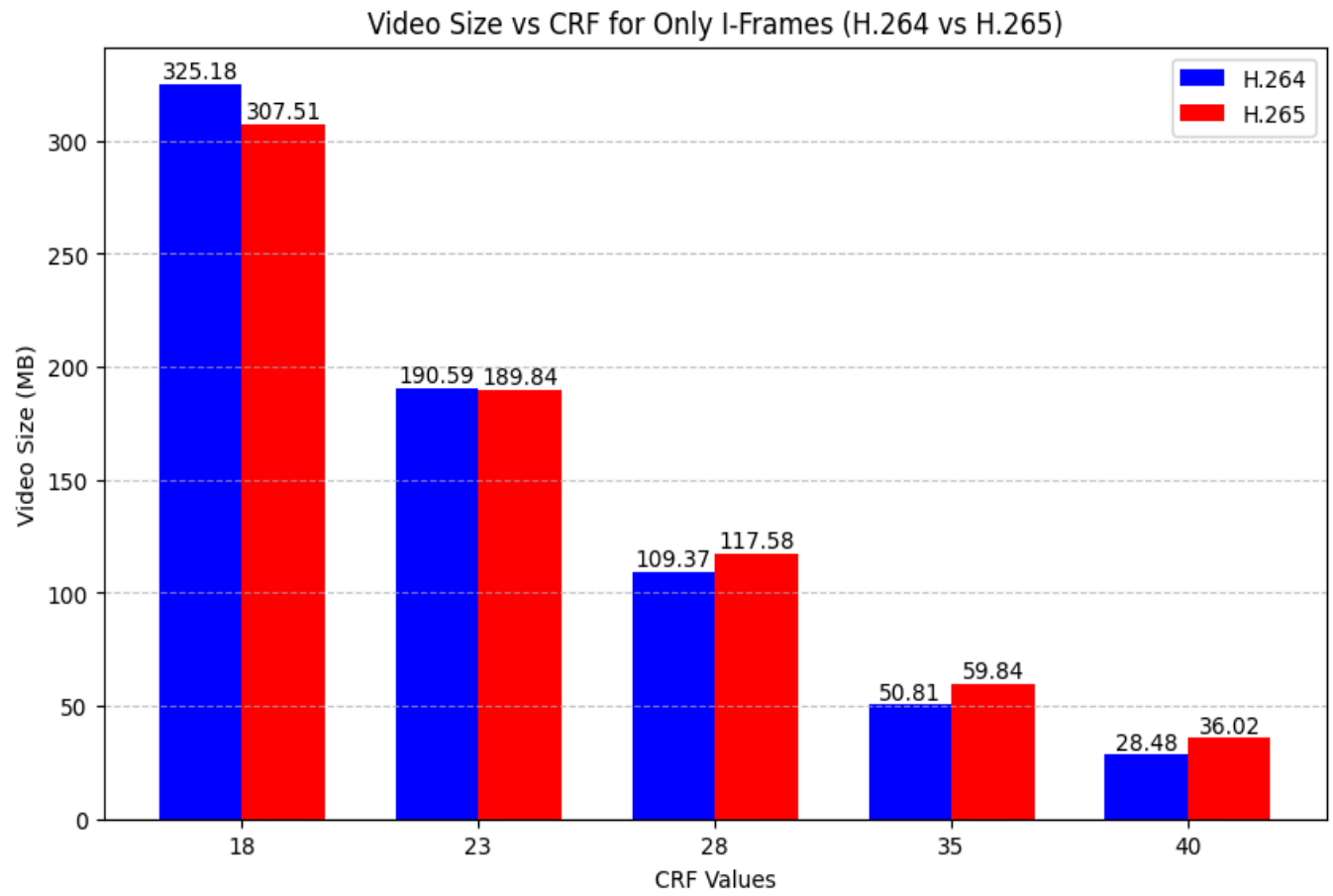


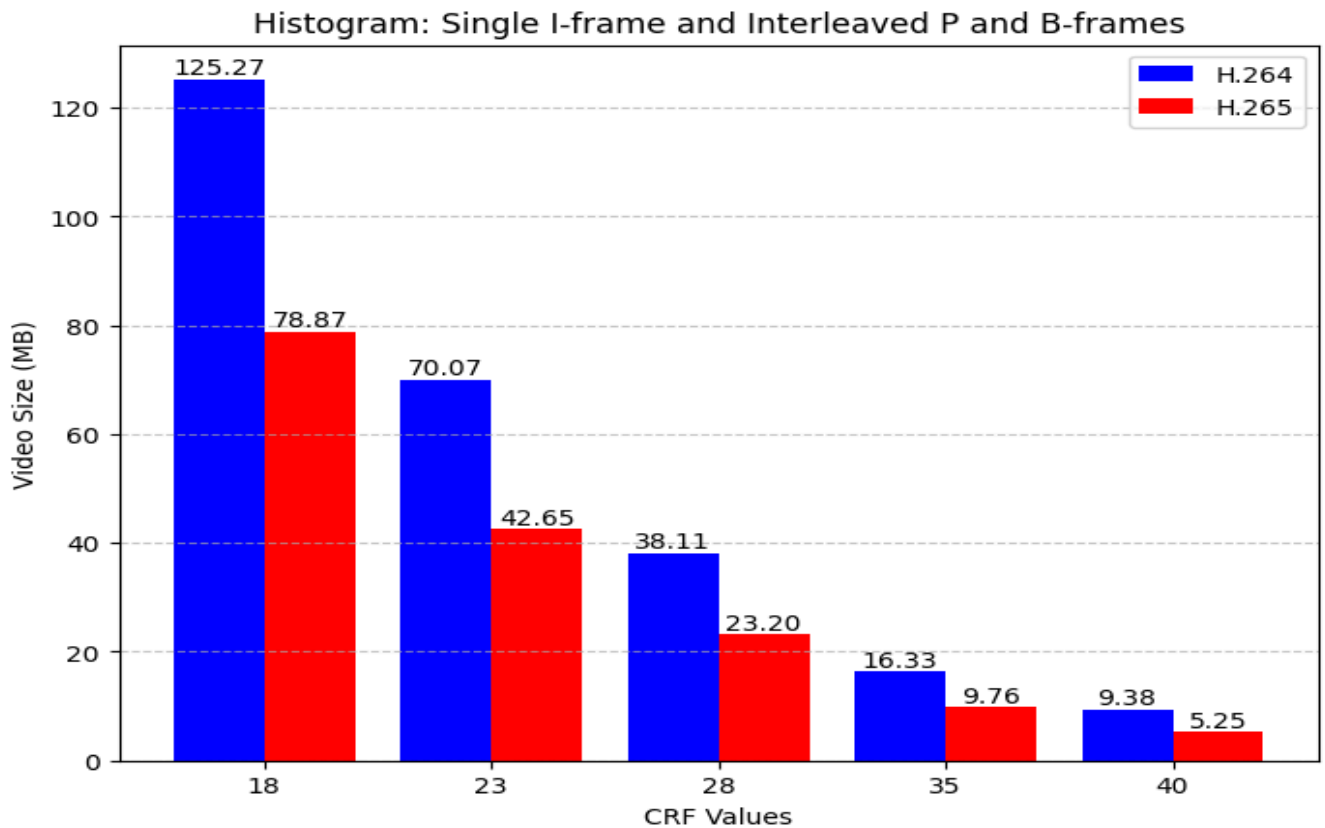
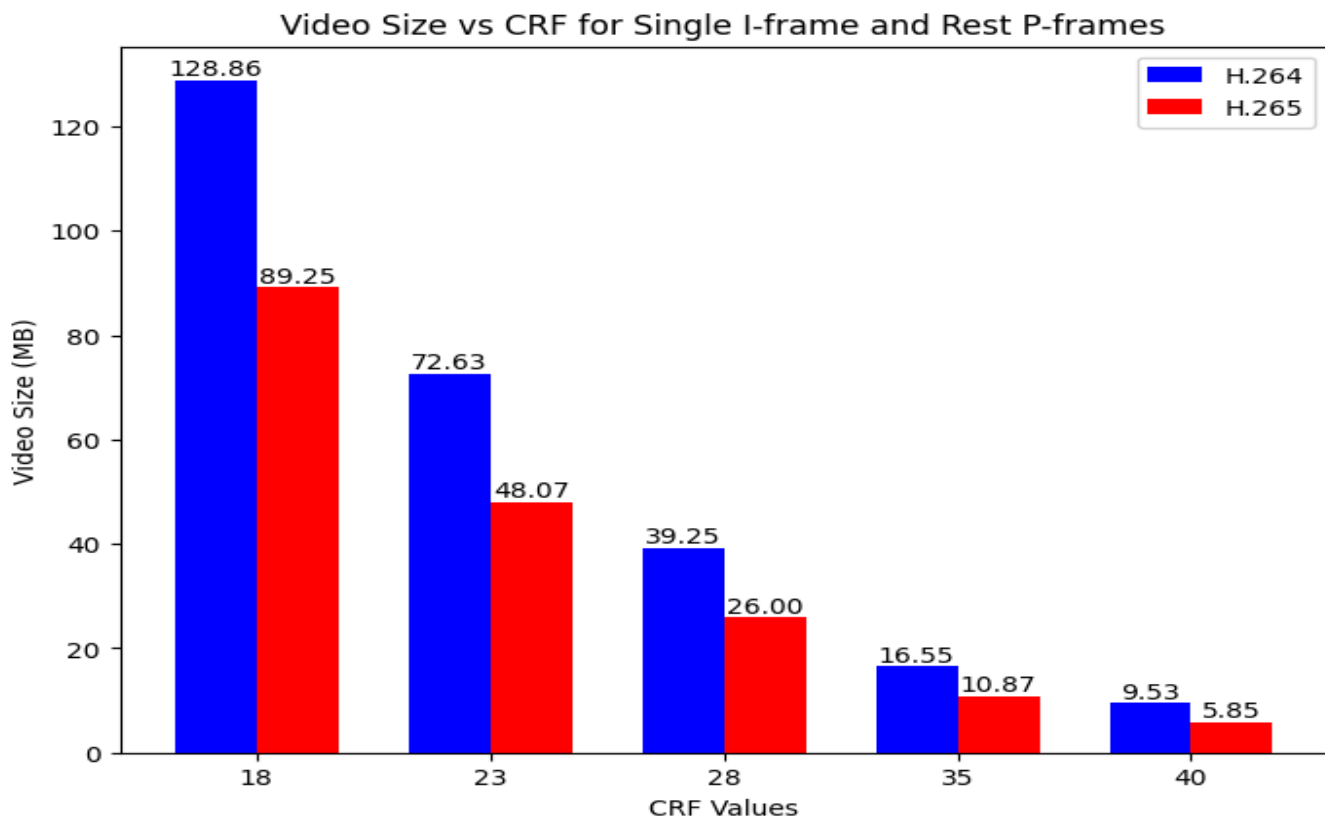
Video link: [dataset_town4](#)





Video link: [dataset_town5](#)

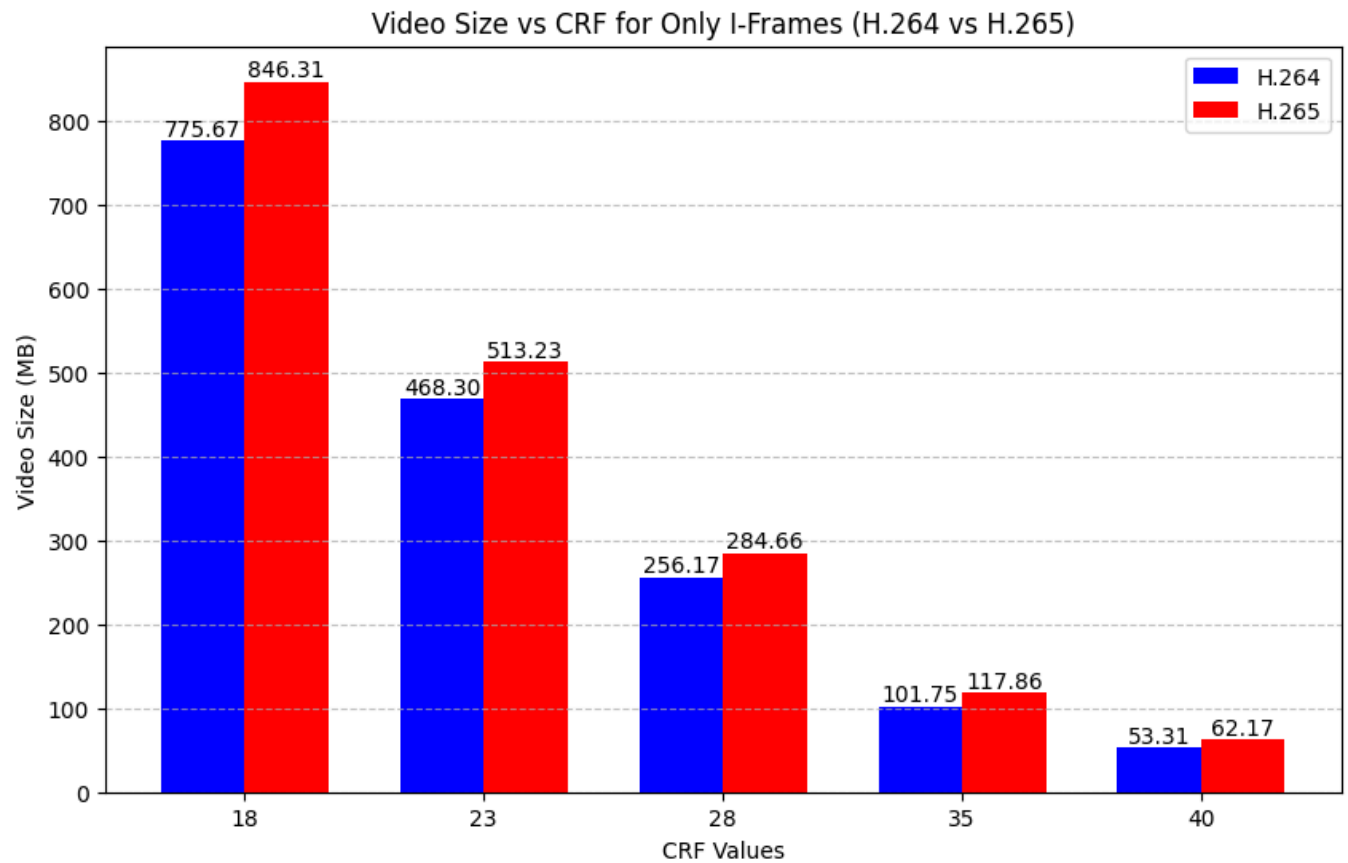


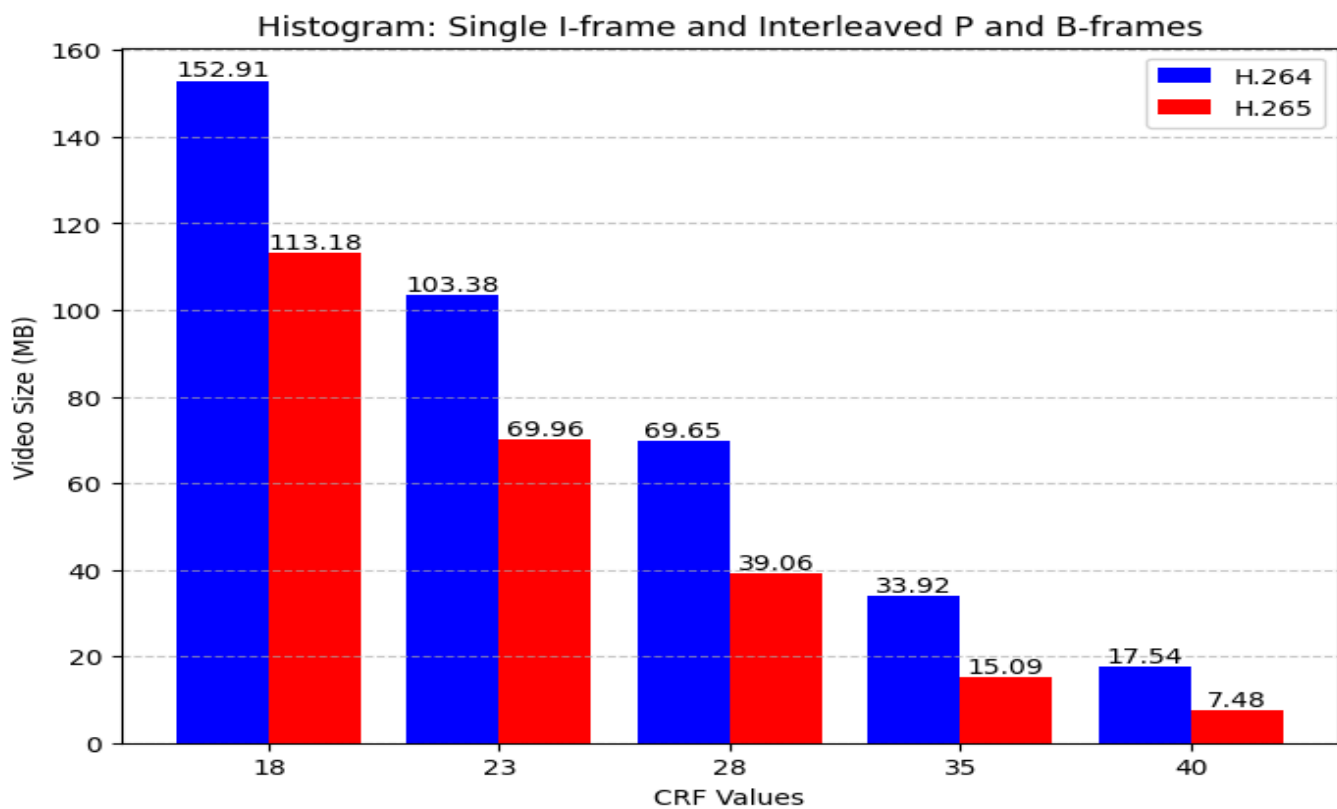
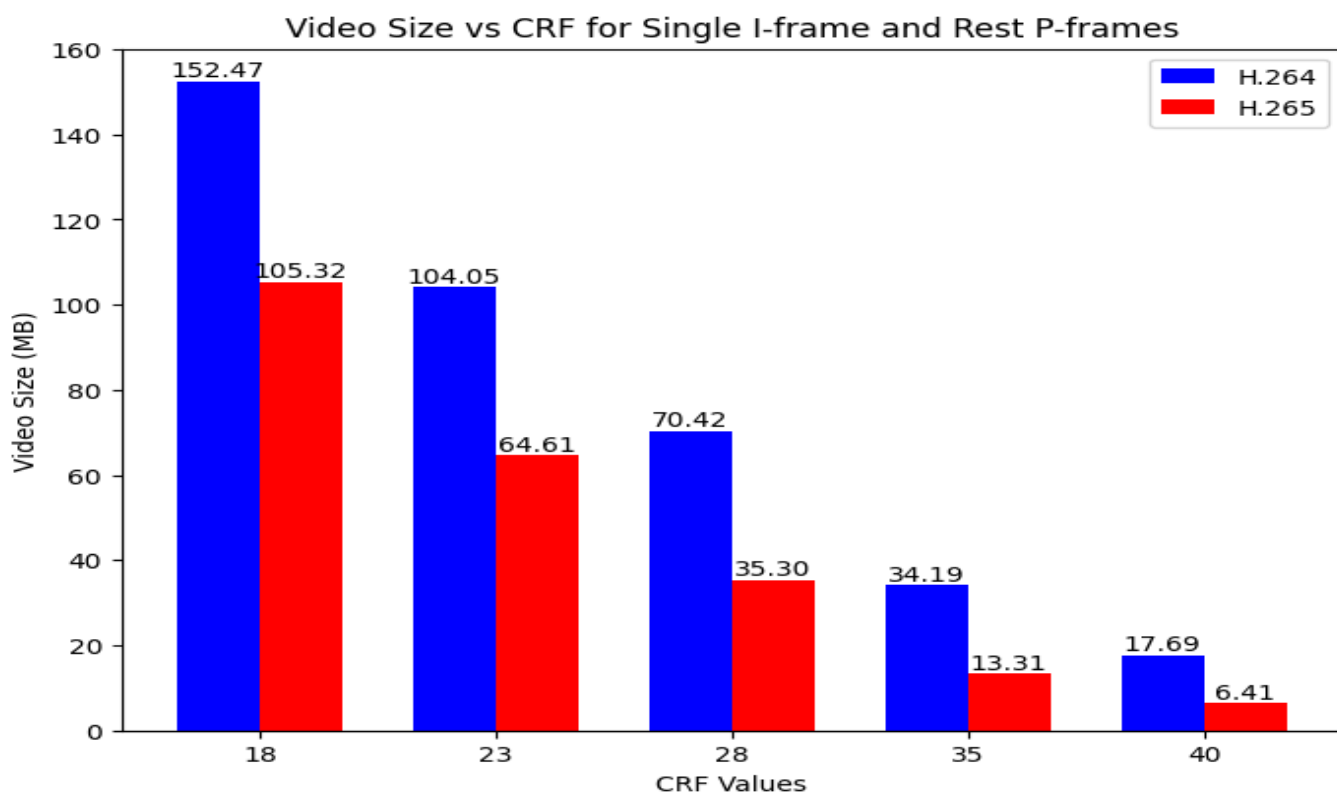


Task Create a descriptive histogram for a static camera video for different frame types

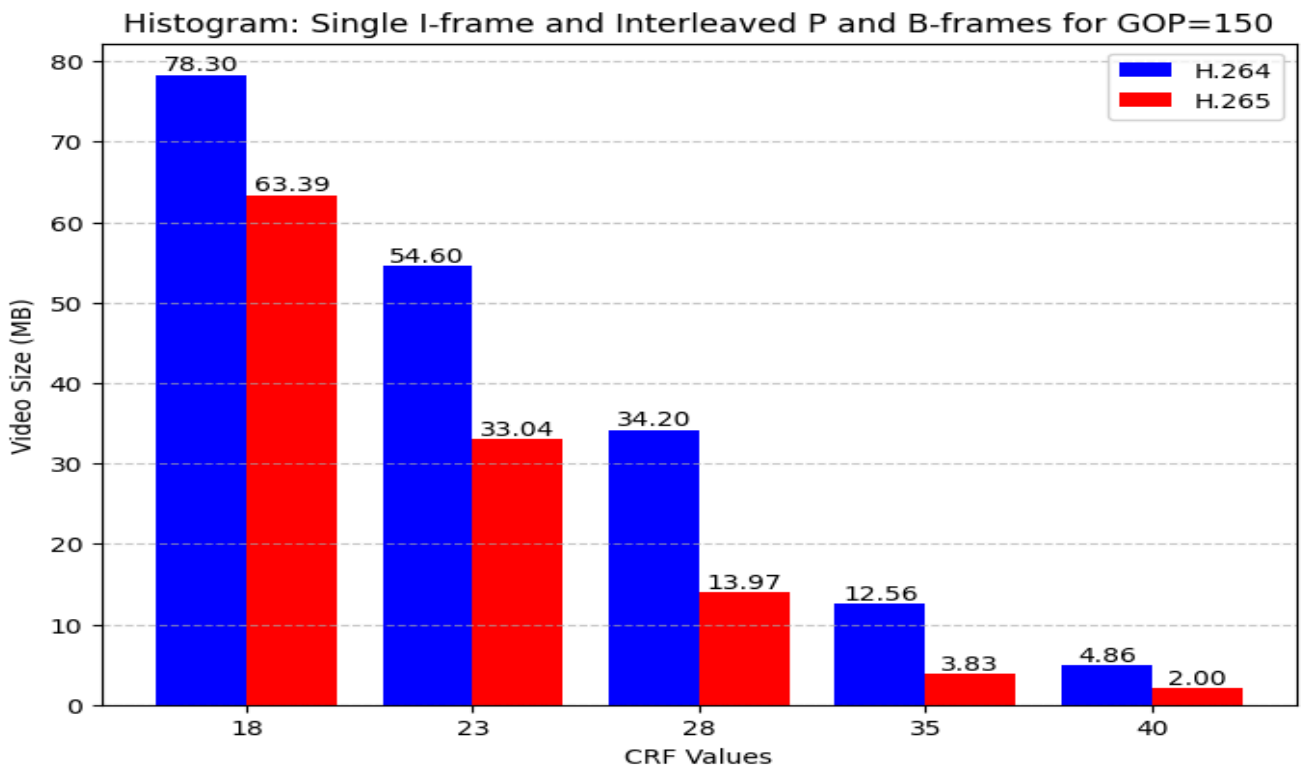
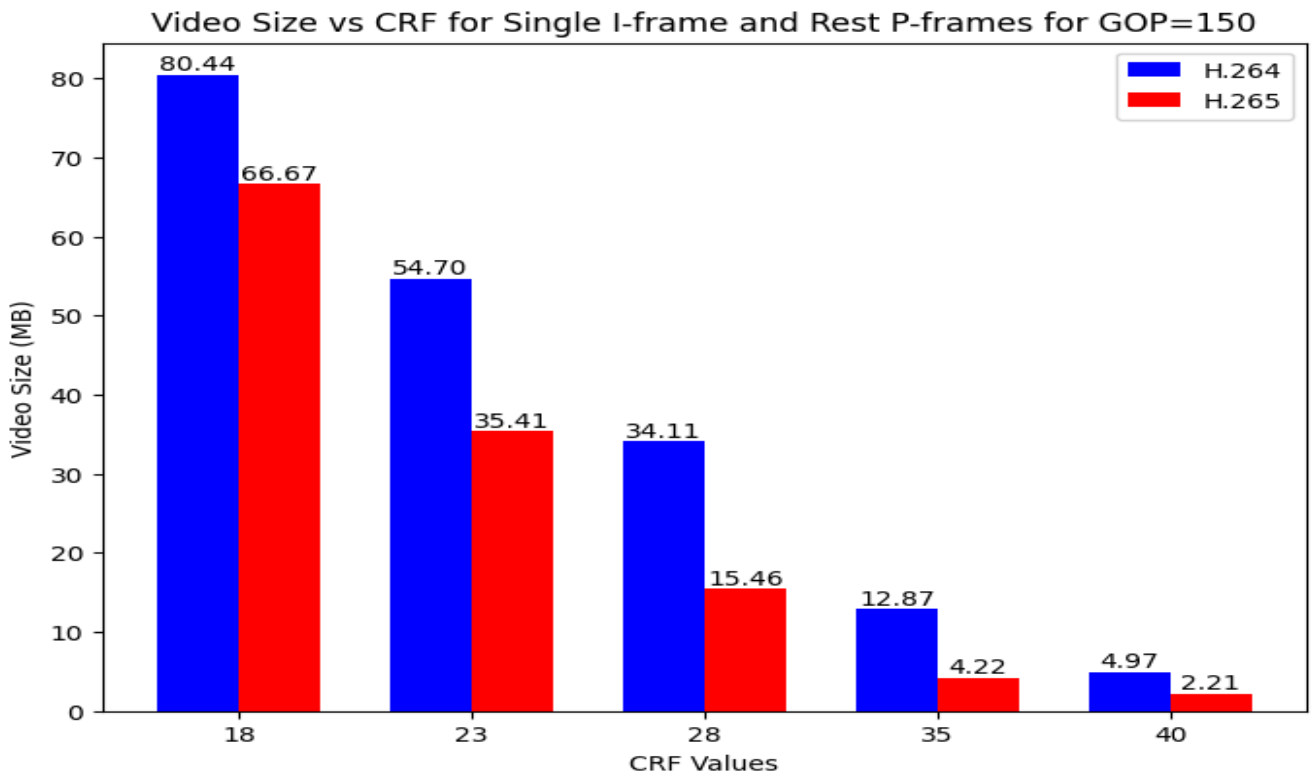
1. Video link: [AITr1cam6_snow](#)

GOP = 15

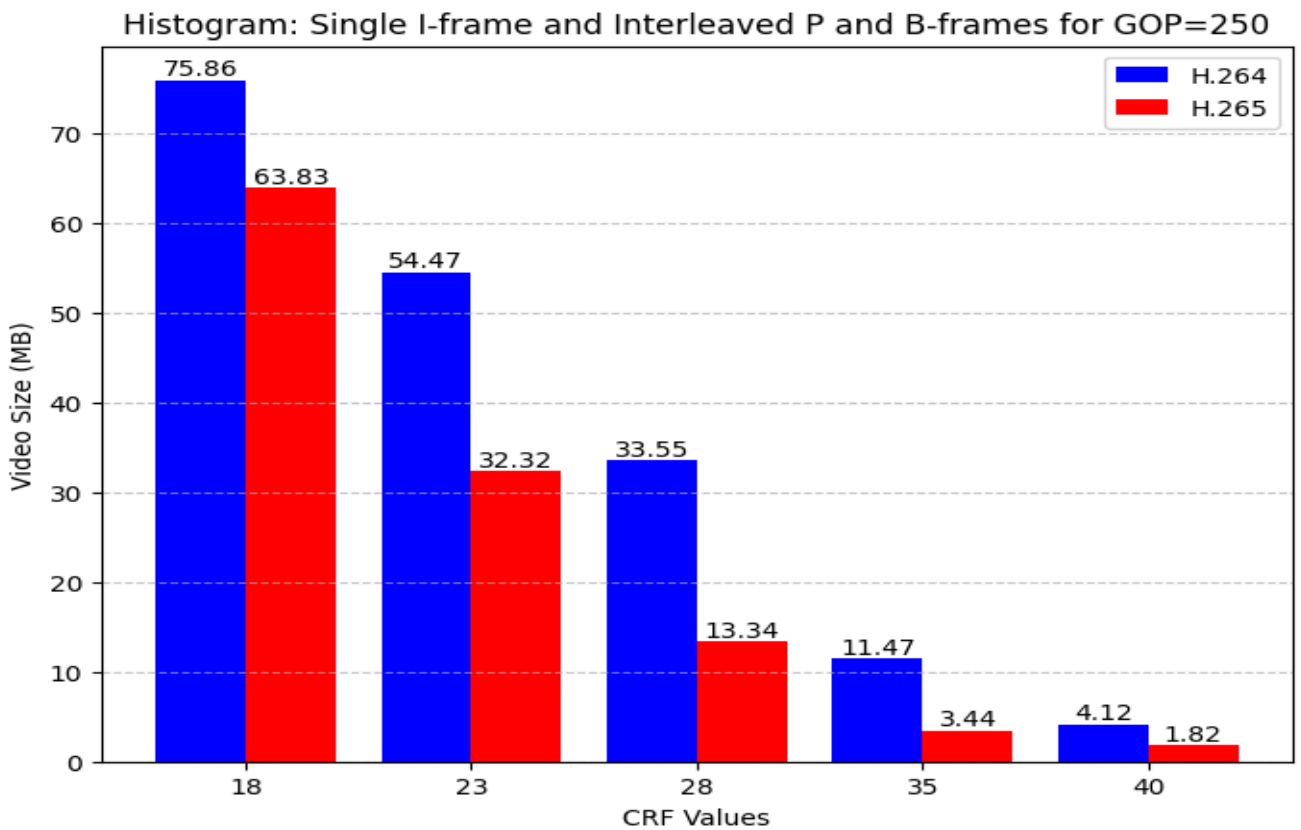
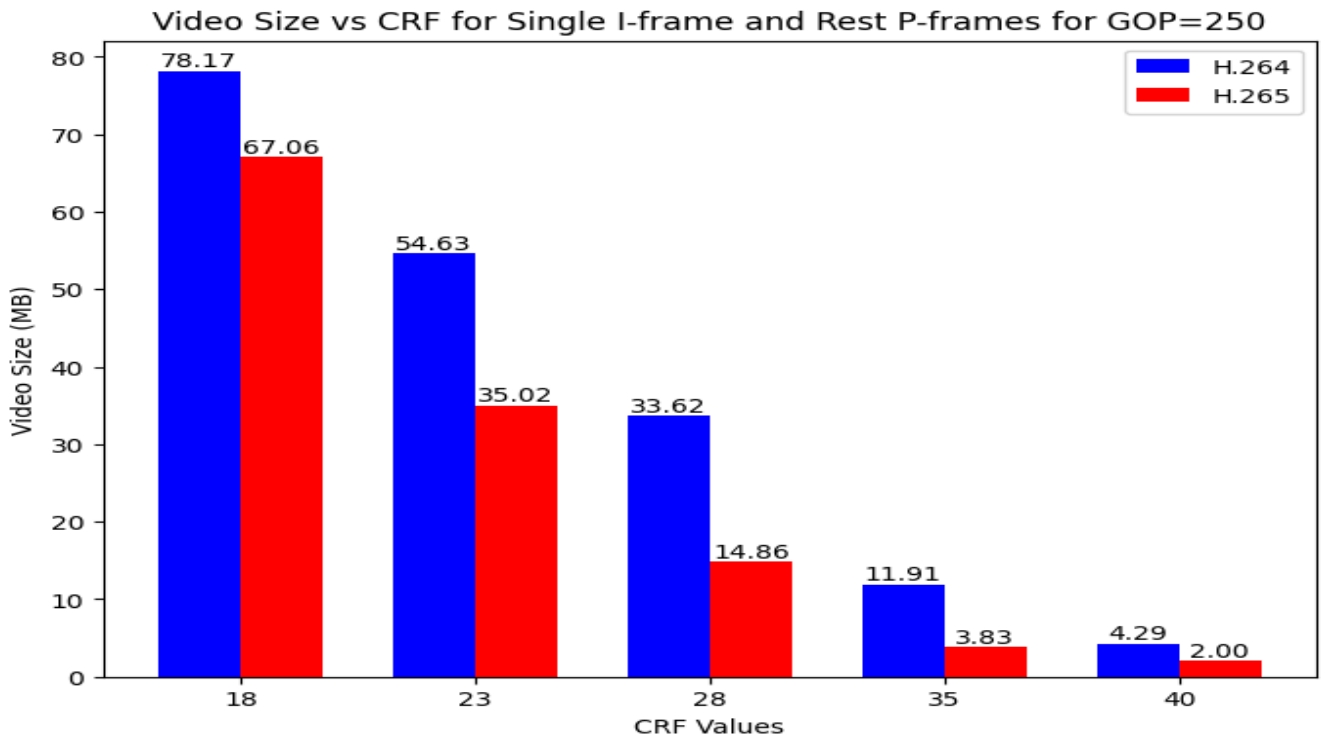




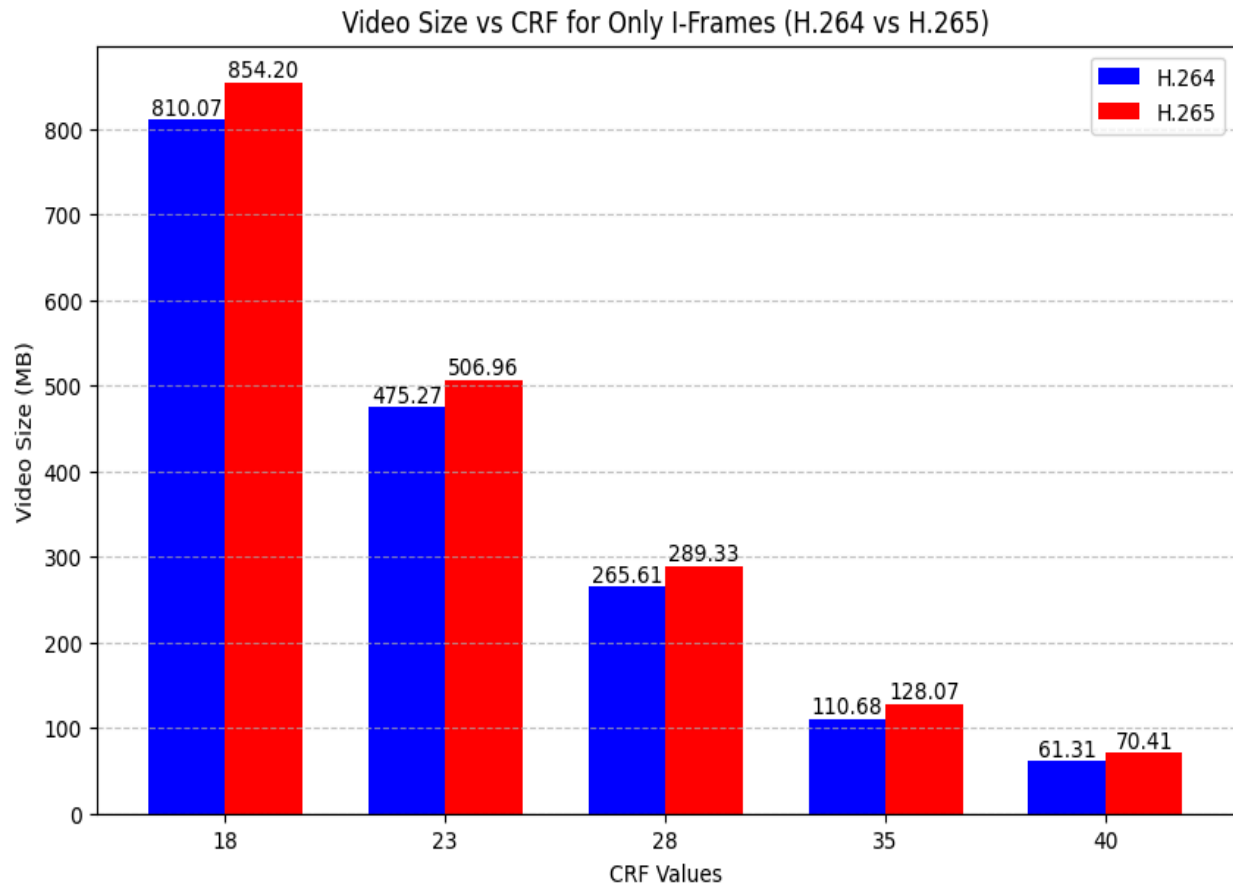
GOP = 150

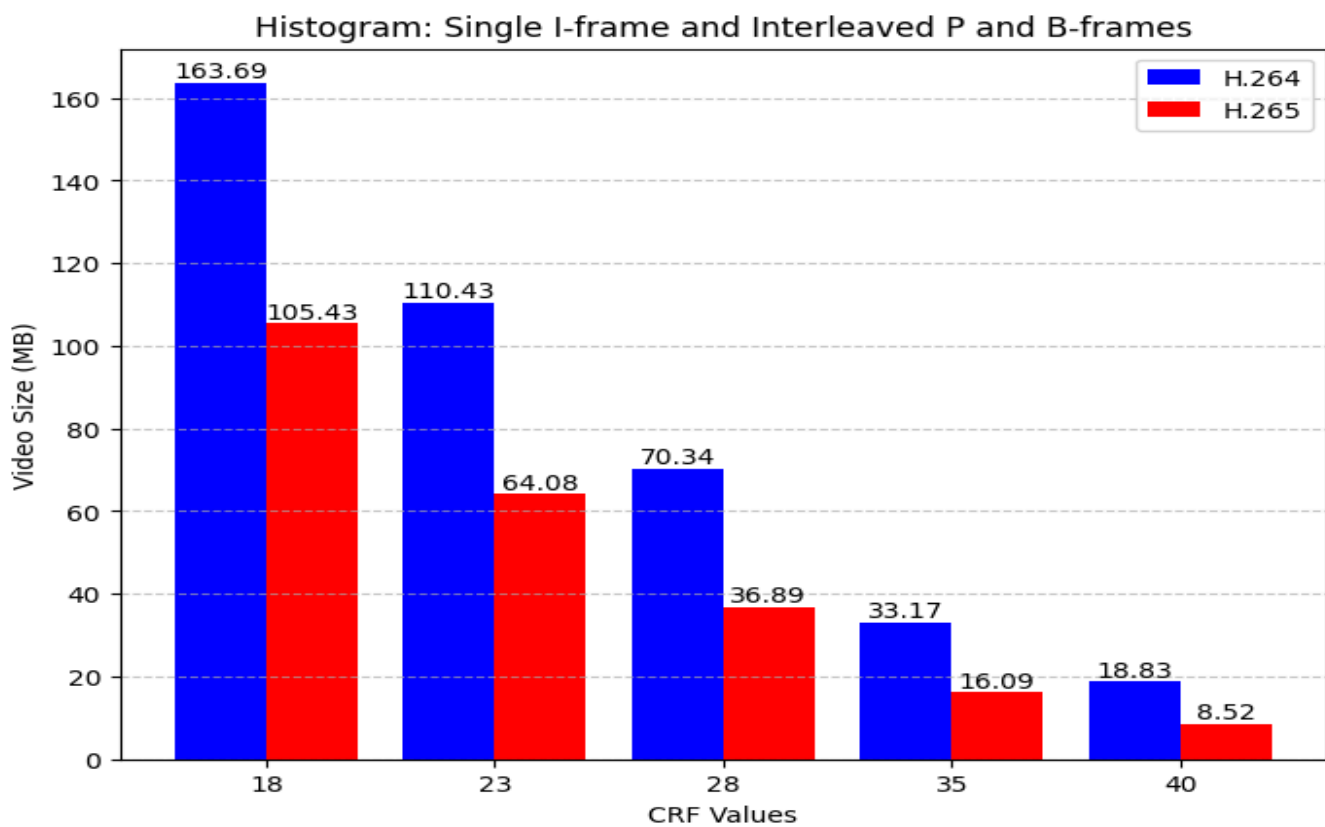
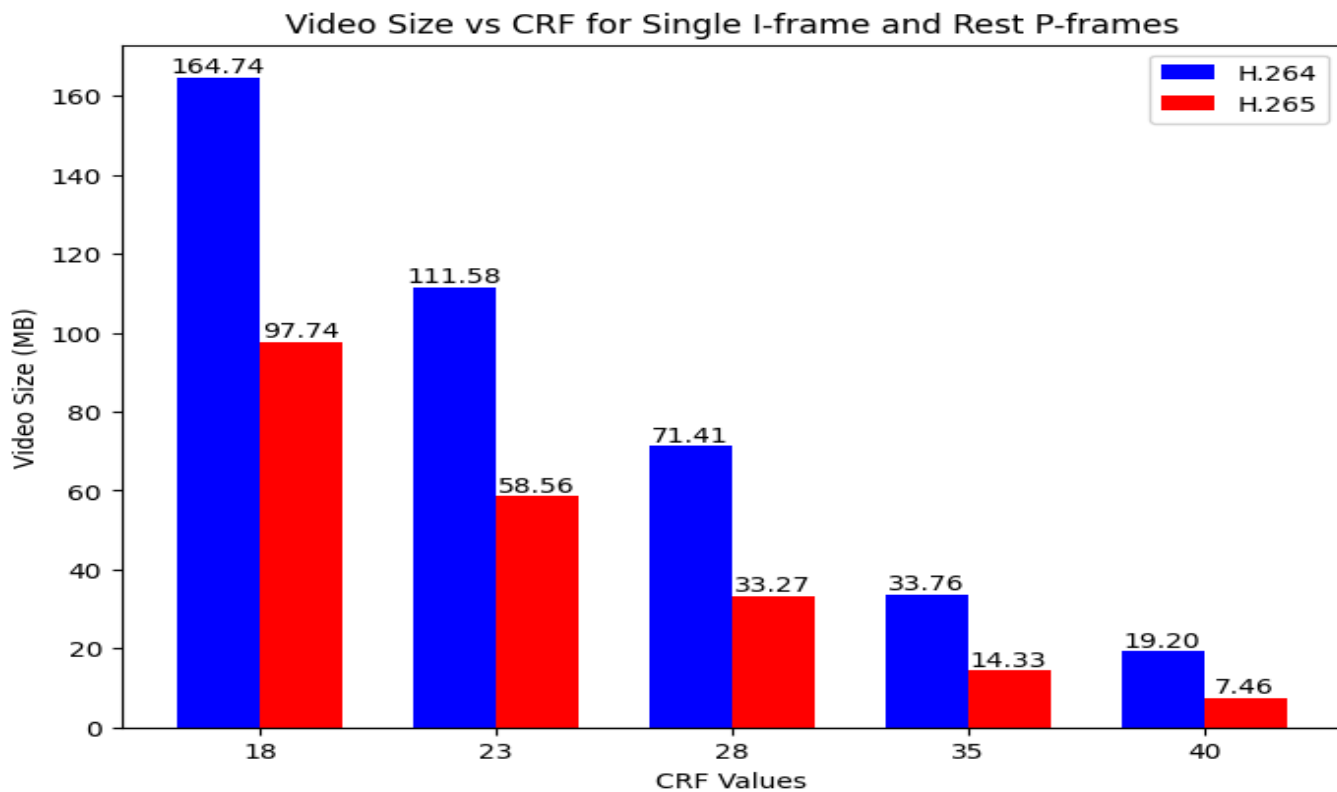


GOP = 250



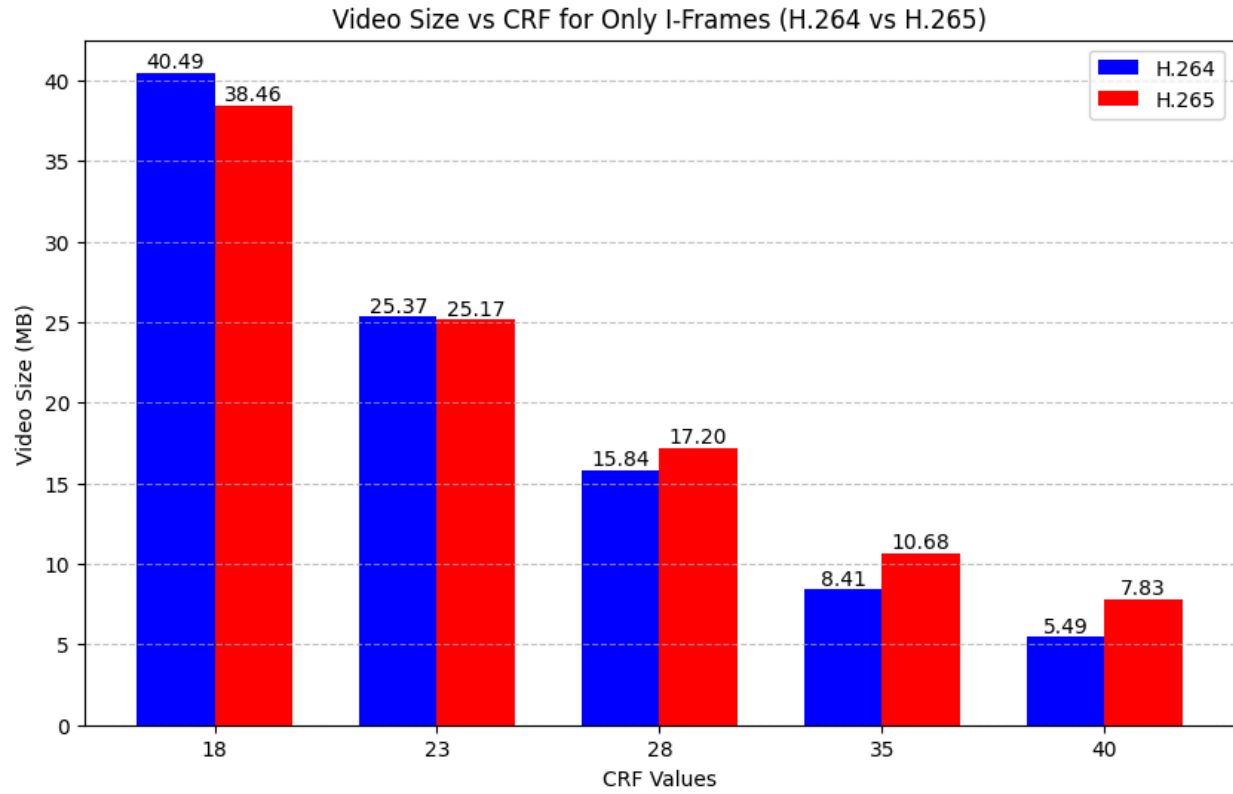
2. Video link: AITr1cam6normal

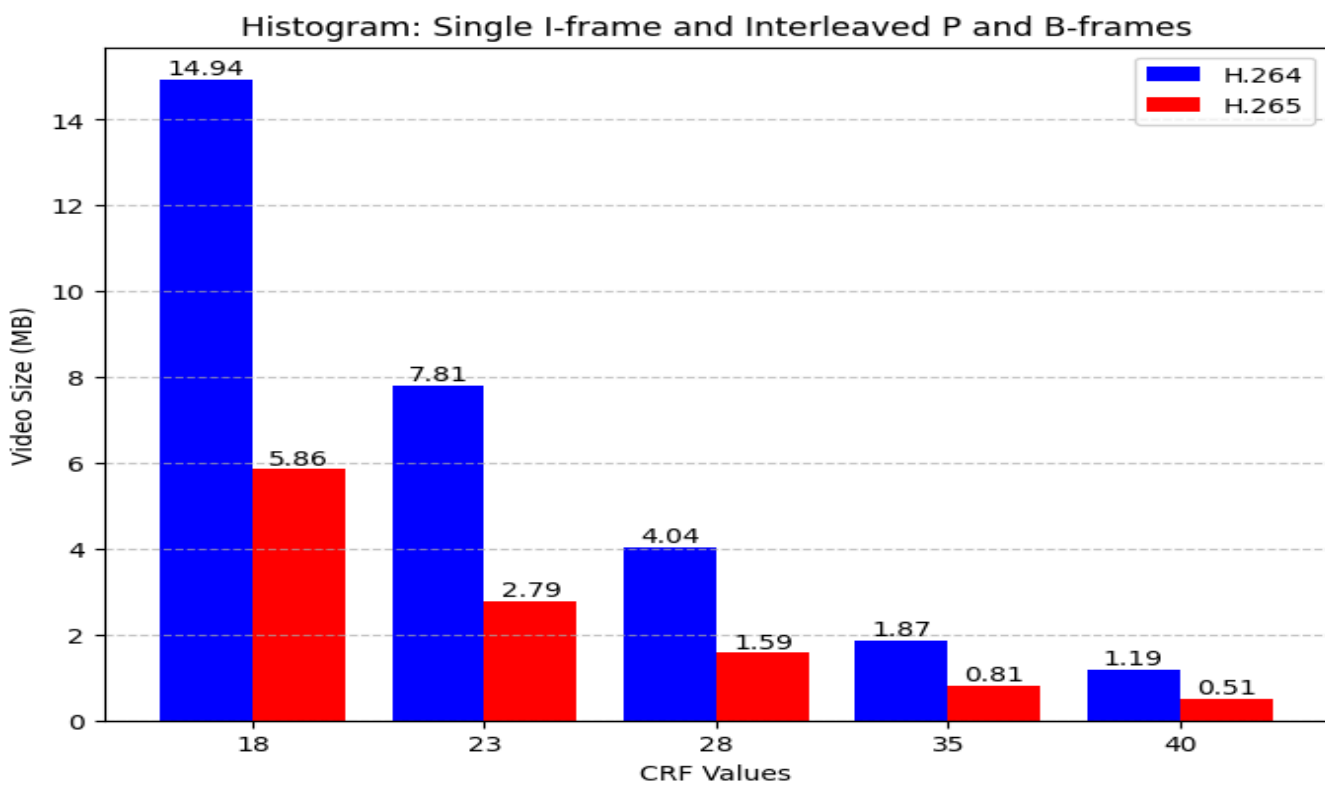
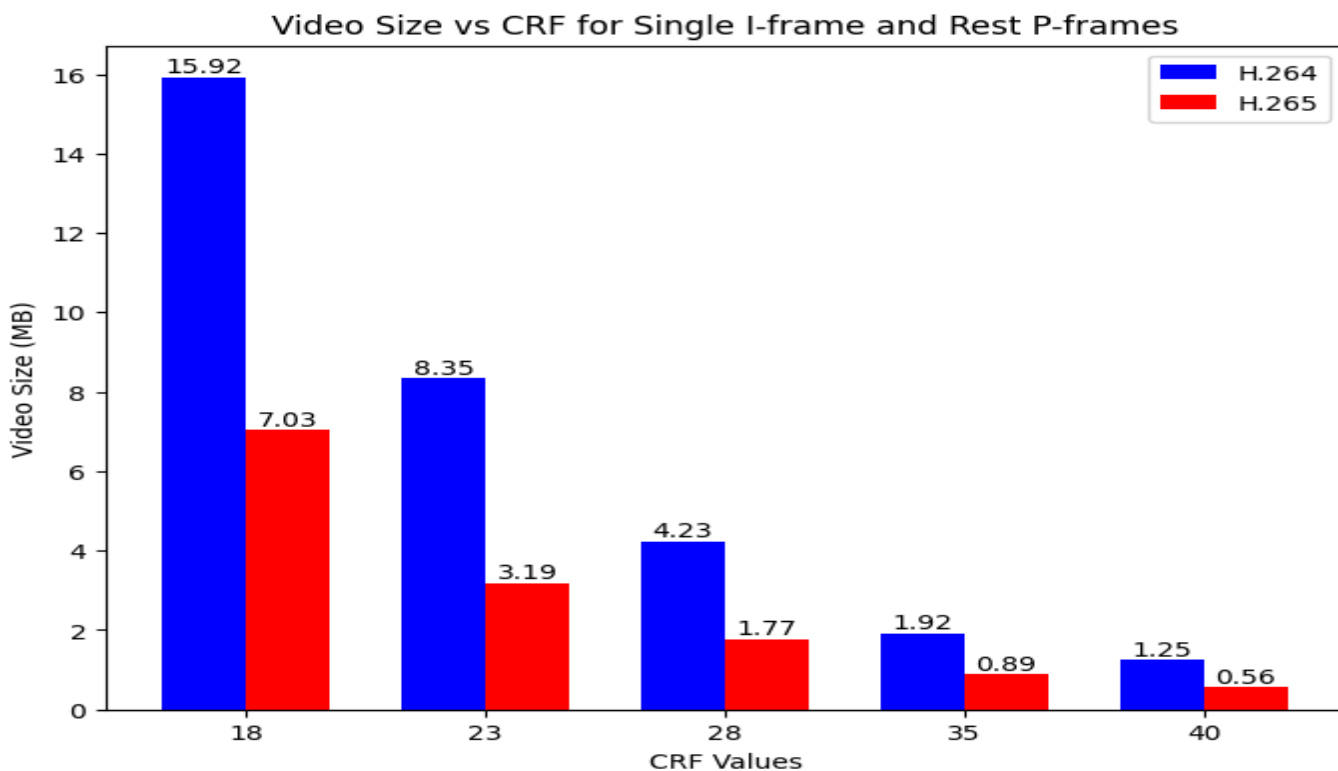




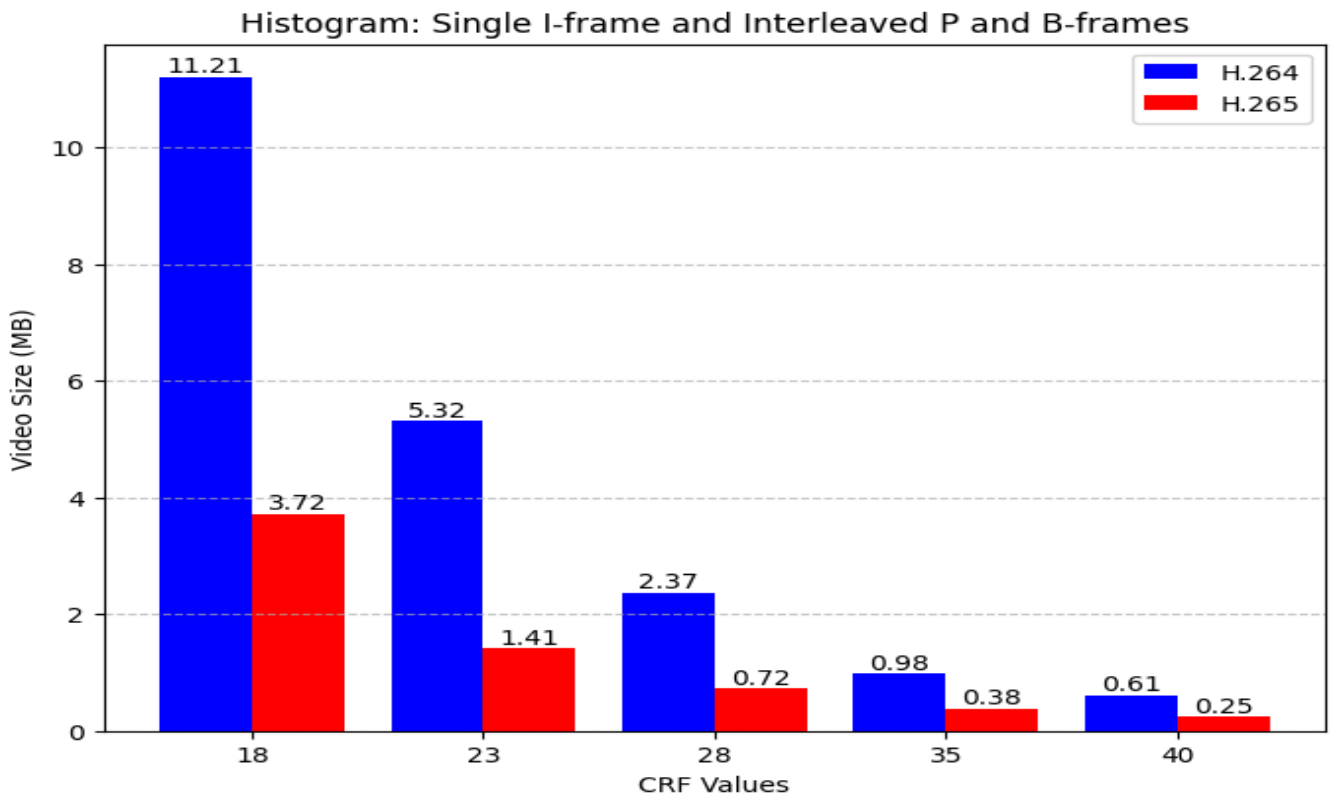
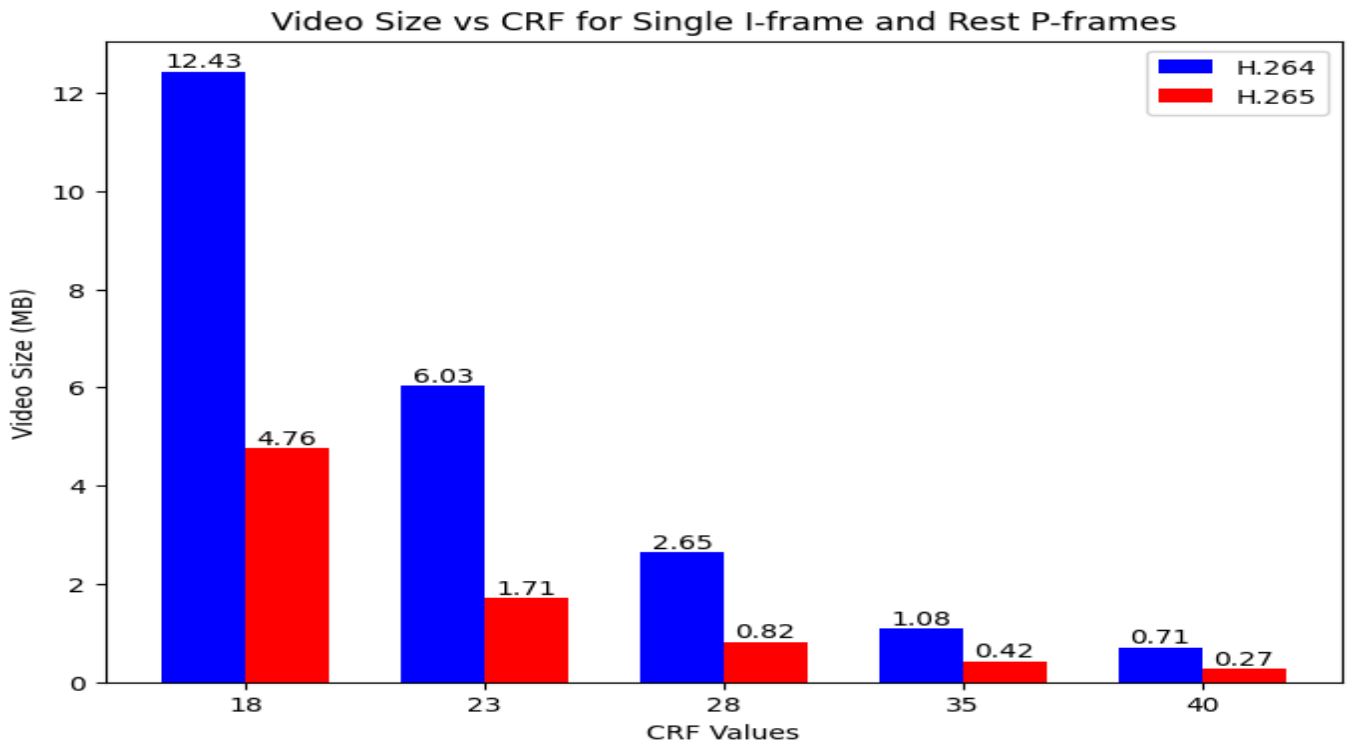
3. Video link: **MVI_39761**

GOP = 15

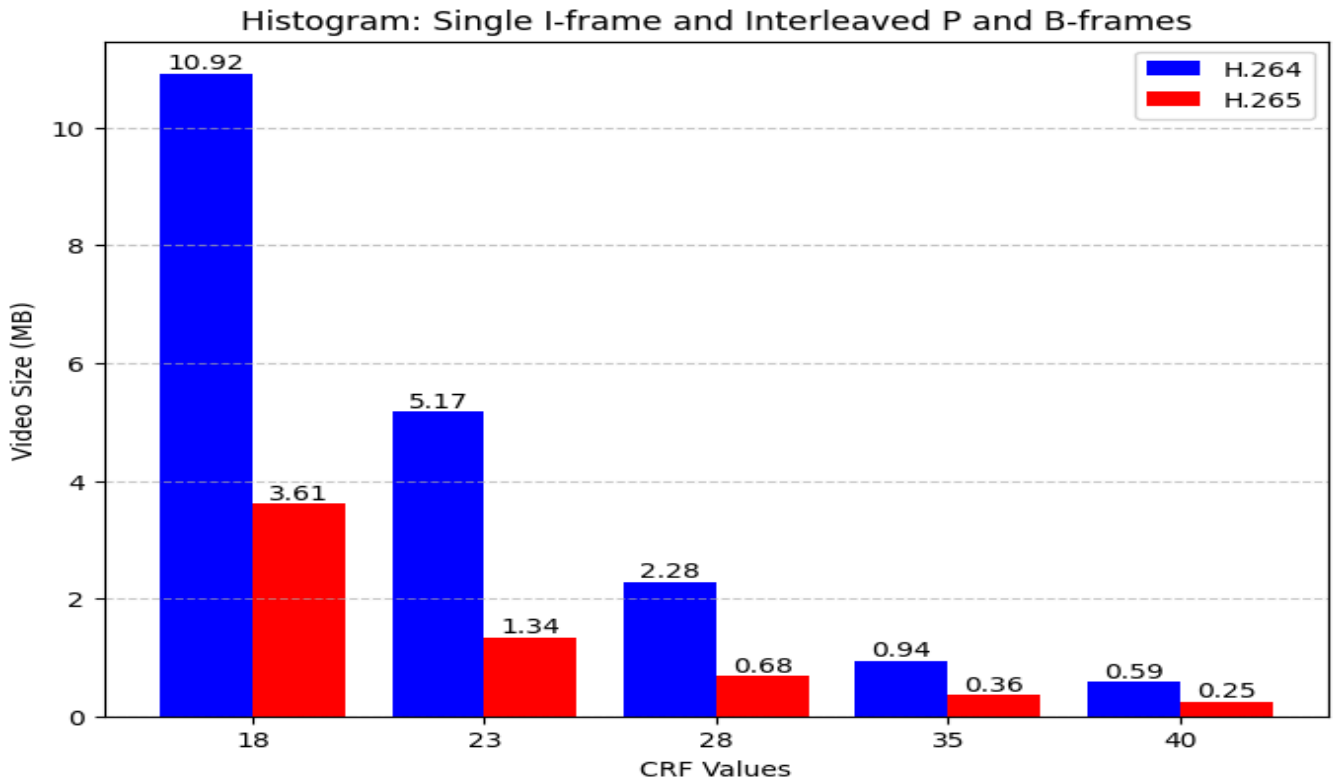
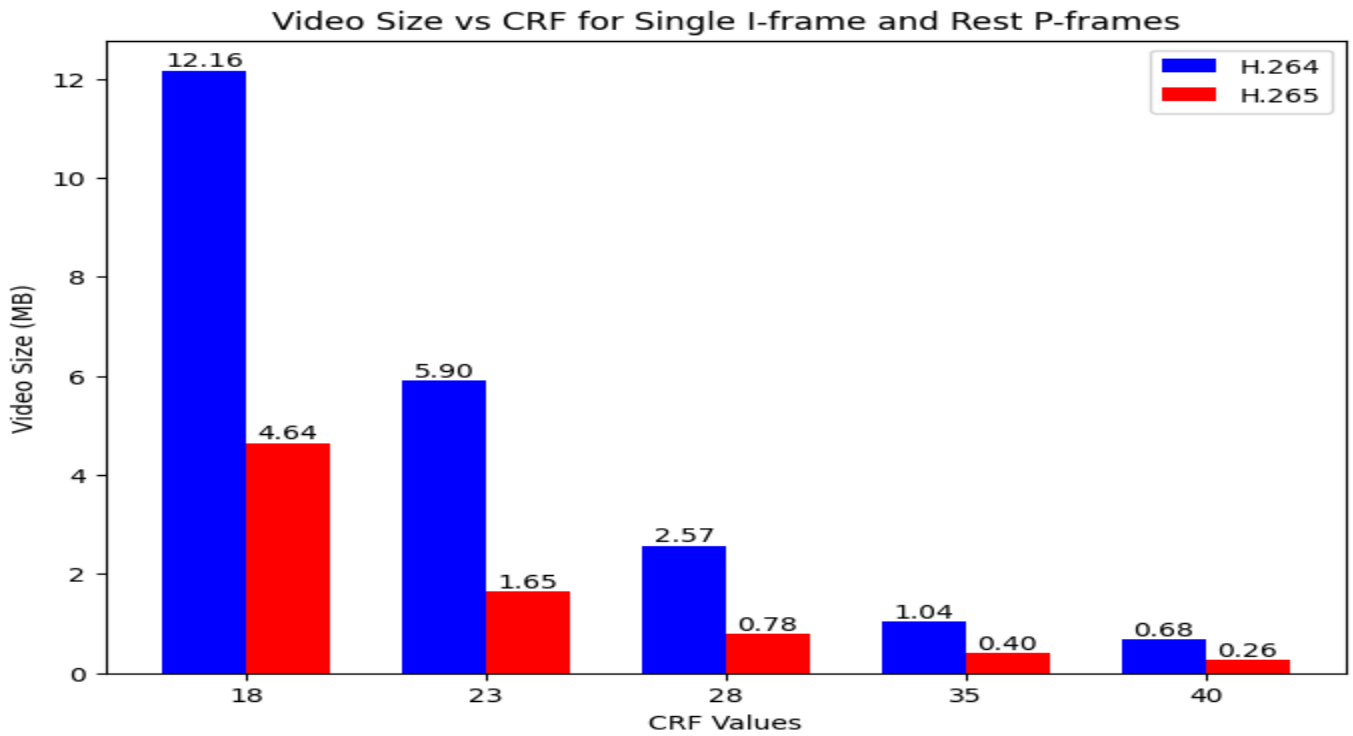




GOP = 150



GOP = 250



Task: Instead of taking crf values, try with qp

Video link: [dataset_feb3](#)

H.264 crf

Encoding techniques/ crf values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Single I and rest P-frames	202.51	104.81	54.09	23.30	12.68
Single I and interleaved B and P-frames	190.61	100.53	53.32	23.29	12.76
Single I and interleaved 3 B's and single P-frames	180.28	96.95	52.73	23.30	12.78

H.264 qp

Encoding techniques/ qp values	18 (in MB)	23 (in MB)	28 (in MB)	35 (in MB)	40 (in MB)
Single I and rest P-frames	192.12	101.42	53.13	23.09	13.02
Single I and interleaved B and P-frames	181.16	96.88	52.04	22.91	13.03

H.264 vbr

Encoding techniques/ vbr values	8000K	4000K	2000K	1000K	500K
Single I and rest P-frames	164.74	84.37	43.17	21.83	10.95
Single I and interleaved B and P-frames	162.77	82.81	42.09	21.21	10.69

AV1 Video Encoding with PyAV Configuration (**svt-av1**)

This script decodes the input video and re-encodes it using the SVT-AV1 codec with golden frames enabled, trying the CRF values you specified (18, 22, 30, 35, 40).

Working:-

1) Decode and Encode Process:

- The script opens the input video file using pyAV
- It extracts the video properties (width, height, frame rate)
- For each frame, it decodes from the input and re-encodes to the output with SVT-AV1

2) Golden Frame Implementation:

- Golden frames are enabled through SVT-AV1 codec parameters
- The `svtav1-params` option with `enable-golden-frame=1` activates the golden frame feature
- Supporting parameters like `hierarchical-levels` and `lookahead` are configured to optimize golden frame usage

3) CRF Testing:

- The script loops through each CRF value you specified
- Lower CRF values (like 18) produce higher quality but larger files
- Higher CRF values (like 40) produce lower quality but smaller files

There are four specific parameters that enable golden frames in the SVT-AV1 encoder:

1. **'svtav1-params': 'enable-golden-frame=1'**: This is the primary parameter that explicitly enables golden frames in the SVT-AV1 encoder. The **enable-golden-frame=1** setting directly tells the encoder to use golden frames.
2. **'enable-overlays': '1'**: This enables overlay frames, which work in conjunction with golden frames. Overlay frames help improve compression efficiency by providing additional reference options.
3. **'hierarchical-levels': '4'**: This sets up a hierarchical prediction structure with 4 levels, which is optimal for golden frame usage. Higher hierarchical levels create a more complex reference structure where golden frames can be better utilized.
4. **'lookahead': '120'**: This extends the lookahead window to 120 frames, allowing the encoder to make better decisions about when to use golden frames. A longer lookahead helps the encoder identify optimal positions for golden frames based on future content.

Video link: [dataset_feb3](#)

Encoding techniques/ crf values	18 (in MB)	22 (in MB)	30 (in MB)	35 (in MB)	40 (in MB)
Single I and rest P-frames	123.81	105.12	71.77	55.21	42.38
Single I and interleaved B and P-frames	No B-frames detected				
Golden frames 'enable-golden-frame=1' 'enable-overlays': '1' 'hierarchical-levels': '4' 'lookahead': '120'	92.35	72.01	42.00	29.80	21.35

Video link: [dataset_town_2](#)

Encoding techniques/ crf values	18 (in MB)	22 (in MB)	30 (in MB)	35 (in MB)	40 (in MB)
Single I and rest P-frames	77.15	63.94	41.33	31.02	23.19
Single I and interleaved B and P-frames	No B-frames detected				
Golden frames 'enable-golden-frame=1' 'enable-overlays': '1' 'hierarchical-levels': '4' 'lookahead': '120'	68.29	54.66	32.41	22.98	16.29

How to verify whether golden frames are really present or not?

AV1 Codec Check

- The detector first checks if the video uses the AV1 codec
- This is foundational because golden frames are specific to AV1

It has a consistent keyframe pattern with the right ratio/interval for golden frame usage

Frame Type Indicators

- The detector looks for specific markers or patterns in the frame data that indicate golden frame usage
- For the SVT-AV1 encoded video, it found "structured reference frames" which is consistent with golden frames

Confidence Assessment

- The detector combines all evidence to determine a confidence level:
 - "Low" - Only basic evidence or insufficient information
 - "Medium" - Strong indicators like consistent frame patterns
 - "High" - Direct references to golden frames in the metadata

Video Link: one_hr

Encoding techniques with crf value = 30

-> **Single I and rest P-frames:** 808.21 MB (GOP size = 161. One I- frame and 160 P frames)

-> **Golden frames:** 784.66 MB (GOP size = 250. One I- frame and 249 P frames)